

UNIVERSITAT POLITÈCNICA DE VALÈNCIA



etsinf

Escola Tècnica Superior d'Enginyeria Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

FARM: A Formal Approach to Reward Machine Generation in Reinforcement Learning

MASTERS' FINAL WORK

Master's Degree in Artificial Intelligence, Pattern Recognition and Digital Imaging

Author: Kevin Martínez García

Tutor: Eva Onaindía De La Rivaherrera

Experimental Director: Ángel Aso Mollar

Course 2024-2025

Resum

En aquest treball, presentem un nou marc per resoldre problemes d'aprenentatge per reforç multiagent seguint una aproximació centralitzada mitjançant l'ús d'un meta-agent supervisor que assigna seqüencialment accions als agents individuals en l'entorn. Aquest supervisor, que s'entrena utilitzant un algoritme estàndard d'aprenentatge per reforç d'un sol agent, aprèn una política que modela implícitament el comportament coordinat de múltiples agents mitjançant l'agregació de les recompenses dels agents individuals en un únic valor escalar. En el treball, explorem diferents estratègies d'agregació de recompenses per tal de millorar el rendiment. A més, distingim entre tasques cooperatives i tasques totalment cooperatives, basant-nos en l'estructura subjacent de recompenses, i analitzem el rendiment de la nostra aproximació en cada cas. Validem el nostre enfocament en diversos entorns utilitzant la biblioteca PettingZoo, que proporciona una àmplia varietat d'entorns d'aprenentatge per reforç multiagent de referència.

Paraules clau: Aprenentatge per Reforç, Aprenentatge per Reforç Multiagent , Entrenament Centralizat, Agregació de Recompenses, PettingZoo, Proximal Policy Optimization (PPO)

Resumen

En este trabajo, presentamos un nuevo marco para resolver problemas de aprendizaje por refuerzo multiagente siguiendo una aproximación centralizada mediante el uso de un meta-agente supervisor que asigna secuencialmente acciones a agentes individuales en el entorno. Este supervisor, que se entrena utilizando un algoritmo estándar de aprendizaje por refuerzo de un solo agente, aprende una política que modela implícitamente el comportamiento coordinado de múltiples agentes mediante la agregación de las recompensas de los agentes individuales en un único valor escalar. En el trabajo, exploramos distintas estrategias de agregación de recompensas para mejorar el rendimiento. Además, distinguimos entre tareas de cooperativas y tareas totalmente cooperativas, basándonos en la estructura de recompensas subyacente, y analizamos el rendimiento de nuestra aproximación en cada caso. Validamos nuestro enfoque en varios entornos utilizando la biblioteca PettingZoo, que proporciona una amplia variedad de entornos de aprendizaje por refuerzo multiagente de referencia.

Palabras clave: Aprendizaje por Refuerzo, Aprendizaje por Refuerzo Multiagente, Entrenamiento Centralizado, Agregación de Recompensas, PettingZoo, Proximal Policy Optimization (PPO)

Abstract

In this work, we present a new framework for addressing multi-agent reinforcement learning problems using a centralized approach based on a meta-agent supervisor that sequentially assigns actions to individual agents in the environment. This supervisor, trained using a standard single-agent reinforcement learning algorithm, learns a policy that implicitly models the coordinated behaviour of multiple agents by aggregating their individual rewards into a single scalar value. In this study, we explore different reward aggregation strategies to improve performance. Furthermore, we distinguish between cooperative and fully cooperative tasks based on the underlying reward structure, and we analyze the performance of our approach in each case. We validate our method across

several environments using the PettingZoo library, which offers a wide variety of reference multi-agent reinforcement learning environments.

Key words: Reinforcement Learning, Multi-Agent Reinforcement Learning, Centralized Approach, Reward Aggregation, PettingZoo, Proximal Policy Optimization (PPO)

Contents

List of Figures

List of Tables

List of algorithms

3.1 Description of a step of the sequential construction of a joint action 19

CHAPTER 1
Introduction

Reinforcement Learning (RL) has emerged as a powerful paradigm for solving sequential decision-making problems across a wide range of domains, from robotics and autonomous driving to large-scale resource optimization. While traditional RL focuses on the interaction between a single agent and its environment, many real-world applications inherently involve multiple agents that must coordinate, cooperate, or compete. This complexity has given rise to the field of **Multi-Agent Reinforcement Learning** (MARL), which studies the learning dynamics of agents in shared environments.

One of the main strengths of Multi-Agent Reinforcement Learning lies in the fact that each agent learns independently based on its interaction with the environment and with other agents. This **decentralized learning process** enables the emergence of complex and coordinated behaviours. In other words, in a MARL setting with n agents, the learning process typically involves optimizing n potentially distinct behaviours. However, this decentralized nature can lead to scalability and coordination issues as the number of agents increases. To address these challenges, **centralized approaches** can be employed. In this work, we focus on one such centralized approach, aiming to reformulate the problem of learning n individual behaviours into the task of learning the behaviour of a single central entity.

As we will see in later chapters, in Reinforcement Learning, agents always learn through reward signals—signals designed to reinforce beneficial behaviours that guide agents toward their goals, while penalizing actions that deviate from them. These reward signals can be formally characterized as functions. In this work, we explore how the structure of reward functions allows us to characterize tasks as **cooperative** or **fully cooperative** and how this distinction influences the performance of our centralized approach. Finally, we will examine how a **centralized entity can process reward vectors** of n components—each corresponding to a different agent—and aggregate them in various ways to produce a single scalar signal that evaluates the overall quality of the centralized entity's behaviour.

1.1 Motivation

The motivation behind this work is threefold. First, we aim to evaluate whether our proposed centralized approach can achieve satisfactory performance in both cooperative and fully cooperative multi-agent environments. These settings pose significant challenges in terms of coordination, and we seek to determine if a single centralized policy can effectively guide multiple agents toward a common goal.

Second, while our centralized approach was initially designed for fully observable environments [**?**], we are particularly interested in assessing its adaptability to partially observable settings. In many real-world scenarios, agents have only limited access to global state information. Therefore, it is crucial to understand whether our method can maintain robust performance under partial observability conditions.

Finally, as we will discuss in later chapters, our centralized approach requires a single scalar reward signal to drive the learning process. However, in multi-agent environments—where each agent receives its own individual reward—it becomes necessary to design aggregation methods capable of transforming reward vectors, composed of all agents' signals, into a single scalar value. This scalar reward is essential for enabling effective learning by the centralized entity.

1.2 Objectives

The main objectives to be addressed throughout the development of this work are as follows:

- To study the theoretical foundations of Reinforcement Learning (RL) and Multi-Agent Reinforcement Learning (MARL).
- To define and identify the key differences between cooperative and fully cooperative multi-agent problems.
- To propose a theoretical framework that encompasses and accurately describes both types of problems, while providing the formal tools required to support our centralized approach.
- To formally construct this centralized approach through the introduction of a supervisor meta-agent.
- To implement, in Python, all necessary components required to develop this centralized approach.
- To design different reward aggregation methods capable of collapsing reward vectors into scalar values that are sufficiently informative to guide the optimization of the supervisor meta-agent.
- To conduct an experimental evaluation using multi-agent environments, with the dual goal of assessing the performance of the centralized approach and comparing the effectiveness of the proposed reward aggregation methods.

1.3 Document structure

Regarding the structure of this document, we begin in Chapter **??** by introducing the theoretical background necessary for the development of this work. We present the formal foundations of Reinforcement Learning and Multi-Agent Reinforcement Learning, and define both cooperative and fully cooperative problems. This chapter also includes a brief review of the current state of the art in both fields, allowing the reader to gain insight into recent developments and potential applications.

In Chapter **??**, we propose a theoretical framework that unifies cooperative and fully cooperative problem settings. We then introduce our centralized approach based on a

supervisor meta-agent, and formally define its operation using the proposed framework. The chapter concludes with a discussion on the need for scalar reward signals in the learning process of the meta-agent, followed by a detailed presentation of the two reward aggregation methods we propose for transforming reward vectors into scalar values.

Chapter **??** focuses on the implementation details of our centralized approach, including the algorithms used to optimize the behaviour of the supervisor meta-agent. We also provide an overview of the multi-agent environments used in our experiments, classifying them as either cooperative or fully cooperative. As will be shown, this project leverages a collection of multi-agent scenarios provided by the PettingZoo API [**?**], a MARL-oriented library maintained by the Farama Foundation.

Finally, in Chapter **??**, we present a detailed account of the experimental process conducted to evaluate the performance of our centralized approach. We analyze how the different reward aggregation methods affect the learning dynamics and final performance of the supervisor. The document concludes with a summary of our findings and a discussion of potential directions for future research.

CHAPTER 2
Background

In this chapter, we present the theoretical foundations necessary to contextualize and support the work developed in this thesis. We begin by introducing the core concepts of Reinforcement Learning (RL) in the single-agent setting, where an agent learns to make sequential decisions by interacting with an environment to maximize cumulative reward. The formalism of *Markov Decision Processes* (MDPs) provides the mathematical basis for this framework, allowing us to model the dynamics of agent-environment interactions.

We then move to the multi-agent setting, where multiple agents interact within a shared environment. To describe these scenarios, we introduce the frameworks of *Multi-Agent Markov Decision Processes* and *Stochastic Games*. These two abstractions are crucial for modeling problems in which agents must achieve varying degrees of cooperation to accomplish a set of common goals. This distinction gives rise to a variety of problem types, such as **fully cooperative** and **cooperative** problems, which we further describe in detail.

Finally, we provide a brief review of the current state of the art in both single-agent and multi-agent reinforcement learning. We highlight notable recent advances and practical applications in domains such as robotics, strategic games, and autonomous systems. This section also outlines some of the open challenges and research directions that motivate the approach and objectives of this thesis.

2.1 Single-Agent Reinforcement Learning

As previously mentioned, single-agent reinforcement learning is based on the sequential interaction between an agent and its environment, where the agent learns to maximize a cumulative reward signal. This interaction is typically modeled as a loop: at each time step t, the agent observes the current state of the environment s_t, selects an action a_t according to a policy π, and executes the action. As a consequence, the environment transitions to a new state s_{t+1}, and the agent receives a scalar reward r_t that reflects the immediate outcome of its action. It is important to note that the agent acts in discrete timesteps which, as we have already seen, we will denote as t.

The policy π, which maps states to actions (either deterministically or stochastically), is the object the agent aims to learn through repeated interaction with the environment. Over time, the agent improves its policy in order to maximize the expected cumulative reward, often referred to as the return. We can provide a mathematical basis for this framework with a *Markov Decision Process* (MDP) [**?**] that we define as follows.

Definition 1 *A Markov Decision Process is a tuple $\langle S, A, \mathcal{R}, \mathcal{T} \rangle$, where S is the discrete state space, A is the discrete action space, $\mathcal{R} : S \times A \times S \to \mathbb{R}$ is the reward function of the agent, and $\mathcal{T} : S \times A \times S \to [0, 1]$ is the transition function, that shows that given a state s and an action a the agent transitions into a next state s' with probability $\mathcal{T}(s, a, s')$.*

According to this definition, an agent who is in a certain state s_t and takes an action a_t, could non-deterministically reach a state s_{t+1} according to the probability distribution defined by the transition function $\mathcal{T}$. At each timestep, the action to be taken is determined by a policy π, which the agent must learn. Formally, this policy is defined as follows.

$$\pi : S \times A \to [0, 1] \tag{2.1}$$

The policy defines a probability distribution over the possible actions a_t the agent can take, conditioned on the current state s_t of the environment. Specifically, $\pi(a \mid s)$ denotes the probability of selecting action a when the environment is in state s. This function is the core object the agent aims to learn, as it fundamentally determines the agent's behaviour and how it interacts with the environment over time. The agent's objective is to maximize the rewards that it perceives in each timestep, according to the returns of the function $\mathcal{R}$. This criterion is therefore the one used to refine and optimize policy π. To achieve this optimization, we define the *discounted reward* or *utility* of a certain timestep t as the amount of reward we can expect to obtain in the timesteps that follow. This reward function can be defined as follows.

$$G_t^\pi = \sum_{k=t}^{T} \gamma^k \mathcal{R}(s_k, a_k, s_{k+1}) \tag{2.2}$$

This quantity G_t^π will therefore be a measure of the expected reward that we would obtain from timestep t, if we follow the actions indicated by policy π. The parameter γ is a scalar in the interval [0, 1] that generates a "damping" of the rewards, depending on how far they are from the current timestep. If gamma takes values close to 1, greater emphasis will be placed on distant rewards while, if its value is closer to 0, greater weight will be placed on more immediate rewards. Note that, in this work, we assume agents operate for a finite number of steps T, thereby assuming a finite-horizon MDP. This contrasts with infinite-horizon MDPs, where the agent can act over an unbounded number of steps. Owing to the nature of the experiments presented in this work, we shall assume from this point forward that all Markov Decision Processes (MDPs) considered operate under a finite-horizon setting. Formally, the objective would be to optimize the policy in such a way that we maximize the expectation from the initial timestep, i.e.

$$\max_\pi \mathbb{E}[G_0^\pi] \tag{2.3}$$

As a summary, and with the aim of clarifying the process, we show in Figure **??** a visual representation of how the agent-environment interaction process works. As can be seen, the agent perceives a certain state s_t from its environment and takes a certain action a_t according to its current policy. On the one hand, the system uses the transition function $\mathcal{T}$ to determine the next state s_{t+1} to which the environment transitions. From the previous state s_t, the action taken a_t and the new state s_{t+1}, a reward value r_t is returned to the agent. This reward value is used during the optimization process to update the policy π in order to maximize the cumulative reward over time. This loop of observing, executing an action and receiving the reward constitutes a complete timestep.

In this way, the agent progressively refines its policy to make better decisions based on its experience.

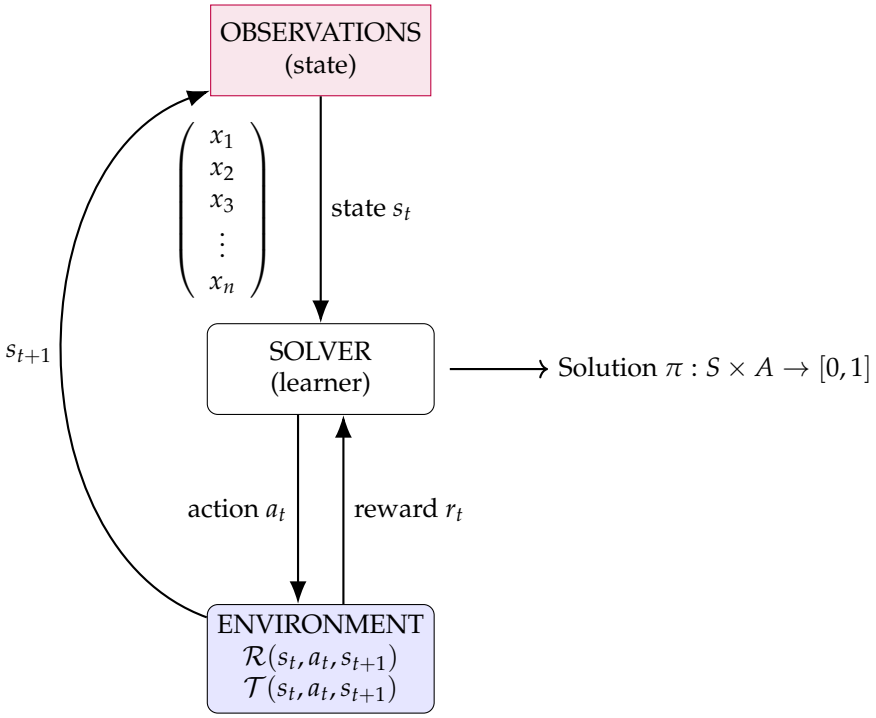


Figure 2.1: Agent-environment interaction loop in Reinforcement Learning

Having introduced the theoretical foundation of reinforcement learning for the single-agent case, we proceed in the next section to expand it to the multi-agent case.

2.2 Multi-Agent Reinforcement Learning

As we saw in Chapter **??**, during our work we will be dealing with multi-agent environments, i.e. environments where several agents coexist and may have shared or potentially individual goals. In order to understand *Multi-Agent Reinforcement Learning* (MARL), we will start with the simplest case, i.e. the extension of MDPs to the multi-agent case, which results in the *Multi-Agent Markov Decision Processes* (MMDPs) [**?**]. Formally, we can define an MMDP as follows.

Definition 2 *A Multi-Agent Markov Decision Process (MMDP) is a tuple* $\langle S, A_1, \ldots, A_m, \mathcal{T}, \mathcal{R} \rangle$*, where* m *is the number of agents,* S *is the finite set of environment states,* $A_i, i = 1, \ldots, m$ *are the finite sets of actions available to the agents, yielding the joint action set* $\boldsymbol{A} = A_1 \times \cdots \times A_m$*,* $\mathcal{T} : S \times \boldsymbol{A} \times S \to [0,1]$ *is the transition function and* $\mathcal{R} : S \times \boldsymbol{A} \times S \to \mathbb{R}$ *is the reward function.*

Intuitively, a Multi-Agent Markov Decision Process is a natural extension of a standard MDP, where instead of a single agent, we now consider m agents interacting with a shared environment. Each agent has its own action space A_i, which allows for potentially diverse modes of interaction with the environment and enables specialization in different tasks. The agent-environment interaction loop remains conceptually similar to the single-agent case. At each timestep t, all agents observe the current state of the environment s_t and select an action $a_t \in A_i$ according to their individual policies π_i. Once all actions have been chosen, they are combined into a joint action $\mathbf{a}_t$, represented as a

tuple of m components—one for each agent. Using the current state s_t and the joint action $\mathbf{a}_t$, the environment transitions, stochastically, to the next state s_{t+1} via the transition function $\mathcal{T}$. Given the transition tuple $(s_t, \mathbf{a}_t, s_{t+1})$, the reward function $\mathcal{R}$ returns a scalar value that reflects the outcome of the agents' joint behaviour in the environment.

A visual representation of this agents-environment interaction loop can be seen in Figure **??**. In this diagram, we propose an environment with three agents that perceive the current state s_t, select their respective actions, and form the joint action $\mathbf{a}_t = (a_1, a_2, a_3)$, which is then used to compute the next state of the environment s_{t+1} through the transition function $\mathcal{T}$. The scalar reward value is also computed from $\mathcal{R}$ and given to all agents in order to further optimize their respective policy π_i.

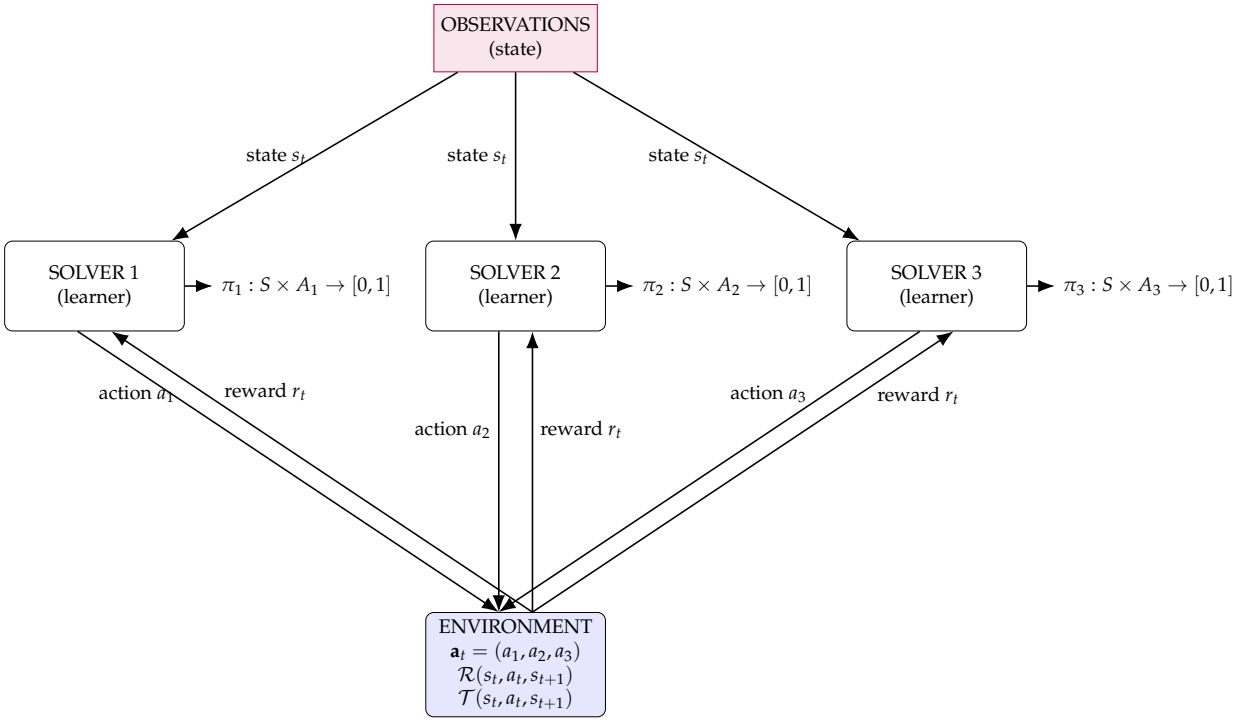


Figure 2.2: Agents-environment interaction loop for MMDPs

Again, each agent's policy π_i can be defined according to Equation **??** and the optimization objective would be to maximize the expectation on the discounted reward from the initial timestep, according to Equation **??**. It is important to note that, in an MMDP, we only use one reward function $\mathcal{R}$ so that, from the joint action of the agents, we **obtain a single shared scalar value that is used during the optimization of all the individual policies**. This can also be seen in Figure **??** where each agent receives the same scalar reward value r_t. It is precisely this characteristic that enables us to define what we refer to as **fully cooperative problems**. In such problems, all agents work together towards a common goal, cooperating to maximize a shared cumulative reward. That is, they are governed by a single, shared reward function, where all agents receive identical rewards based on the overall performance of the team.

An example of such a problem is a modified version of the classic game of Pong in which the paddles must cooperate to keep the ball in play for as long as possible. If we consider each agent as one of the paddles, both share the same objective: preventing the ball from leaving the playing field. Each agent could receive a positive reward for every timestep the ball remains in play, and a negative reward if the ball escapes. The agents' goal would therefore be to maximize their cumulative reward by avoiding nega-

tive outcomes—that is, by coordinating their actions to keep the ball in play for as long as possible, ideally indefinitely or until the game ends.

This abstraction allows us to capture some of the environments we will be working with in our experimental phase. It does not, however, account for scenarios in which agents share a common global goal but may pursue it through more individualistic behaviours. These are referred to as **cooperative problems**. Unlike fully cooperative settings—where all agents receive identical rewards—cooperative problems permit individual reward functions that are aligned with the overall task but may differ across agents. This allows for autonomous agent behaviours that contribute to the collective objective without requiring uniformity in actions or incentives. All in all, agents do not share the exact same reward but benefit from the success of the group and while they may act independently, cooperation always leads to improvements in global performance. To account for this, we proceed to introduce *Stochastic Games* (SGs) [**?**] as follows.

Definition 3 *A Stochastic Game (SG) is a tuple* $\langle S, A_1, \ldots, A_m, \mathcal{T}, \mathcal{R}_1, \ldots, \mathcal{R}_m \rangle$ *where m is the number of agents, S is the finite set of environment states,* $A_i, i = 1, \ldots, m$ *are the finite sets of actions available to the agents, yielding the joint action set* $\mathbf{A} = A_1 \times \ldots A_m$, $\mathcal{T} : S \times \mathbf{A} \times S \to [0, 1]$ *is the state transition probability function, and* $\mathcal{R}_i : S \times \mathbf{A} \times S \to \mathbb{R}$, $i = 1, \ldots, m$ *are the reward functions of the agents.*

Similar to an MMDP, in an SG, we have m agents interacting with their environment, each with its own set of actions A_i. However, **each agent has its own reward function $\mathcal{R}_i$, which potentially allows them to receive different reward signals**. The agent-environment interaction loop is again similar to the previous case. The agents perceive the state of the environment s_t, decide on the action to be taken $a_t \in A_i$ according to their individual policies π_i, and the joint action $\mathbf{a}_t$ is constructed. From the state s_t and this joint action $\mathbf{a}_t$, we use the transition function $\mathcal{T}$ to non-deterministically transition to the next state s_{t+1}. Given the tuple $(s_t, \mathbf{a}_t, s_{t+1})$, each agent receives an individual reward according to its own reward function $\mathcal{R}_i$. This individual reward is used to optimize the agent's policy π_i. This agent-environment interaction loop is the same as the one proposed in Figure **??**, but in this case, agents receive a different reward value according to their respective reward function $\mathcal{R}_i$.

A representative example of a cooperative problem is the *Knights, Archers & Zombies* [**?**] environment from the *Butterfly* category in PettingZoo. A visual representation is provided in Figure **??**. In this environment, two types of agents, knights and archers, must work together to eliminate an ongoing wave of zombies and prevent them from reaching the bottom of the map. Each agent receives a positive reward whenever it successfully eliminates a zombie, and the episode terminates as soon as any zombie manages to evade the agents and reach the bottom boundary. This environment exemplifies a cooperative setting: although all agents aim to achieve a shared objective, that is, ensuring that no zombie escapes, they receive individual reward signals based on their personal contributions. This reward structure fosters the emergence of cooperative strategies, where each agent may develop an individual strategy to eliminate zombies, while still needing to cooperate to prevent the game from ending.

In summary, we have introduced the MMDP framework to characterize fully cooperative problems, and Stochastic Games to formalize cooperative problems. However, it would be beneficial to adopt a theoretical framework that unifies both types of interactions. As we will see in Chapter **??** we will need to operate effectively in both fully cooperative and cooperative settings, and thus it becomes essential to develop a general framework that accommodates both types of problems. Furthermore, the frameworks discussed so far assume that agents have full observability of the environment, i.e., they

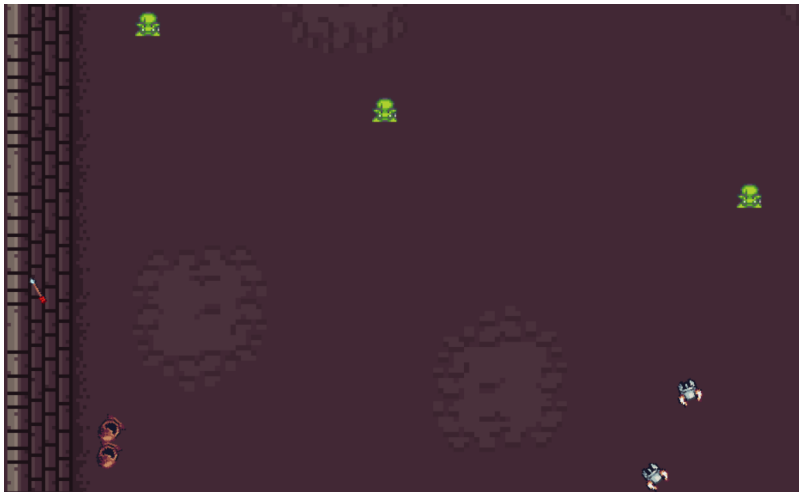


Figure 2.3: Knights, Archers & Zombies environment

can perceive the complete state s_t at each timestep. In contrast, the environments used in our experimental phase will involve partial observability, where agents only have access to limited local observations of the true state. Therefore, we require a unified framework that can model both fully cooperative and cooperative problems, while also handling environments with partial observability. In the next chapter, we introduce such a framework, providing its formal definition and exploring in depth the notion of partial observability and its implications.

2.3 Applications & State of the art

Reinforcement Learning has experienced remarkable growth in recent years, establishing itself as a powerful tool for solving complex tasks that require sequential decision-making. Its ability to learn optimal strategies in dynamic and partially observable environments has enabled its application across a wide range of domains in both research and industry. For example, in the field of Robotics, RL has proven particularly valuable results for physical control tasks, enabling agents to acquire complex motor skills such as walking [**?**], object manipulation [**?**], or even drone flight [**?**]. These abilities are often learnt through trial-and-error interactions in simulated environments before being transferred to the real world.

Another prominent area of application is network and computing systems optimization, where RL is used for dynamic resource allocation, traffic control in communication networks, and automatic tuning of cloud computing systems [**?**]. Another promising application is the energy consumption optimization of Cloud Radio Access Networks (C-RANs), which have become a key component in enabling next-generation 5G wireless communications. By leveraging Deep Reinforcement Learning (DRL) techniques to formulate the resource allocation problem as a convex optimization task, researchers have achieved significant energy savings while still meeting the quality-of-service requirements of both clients and end users [**?**].

Large Language Models (LLMs) have emerged as one of the most impactful branches of artificial intelligence today. Publicly accessible models such as *ChatGPT* or *DeepSeek* have proven to be extremely versatile tools across a wide range of academic and professional contexts. A critical challenge in the development of these models—trained on massive datasets—is to ensure that their outputs are aligned with the values and preferences of human users, particularly during the post-training phase. In this context, Reinforcement

Learning plays a central role in fine-tuning LLMs to produce responses that are not only coherent and accurate but also helpful, safe, and aligned with human intent. One of the most prominent methods used for this purpose is *Reinforcement Learning from Human Feedback* (RLHF), which has been notably applied in the development of models such as ChatGPT [**?**]. RLHF allows the model to adapt its behaviour based on preferences expressed by human annotators, making it more responsive to user expectations in real-world interactions.

Other approaches—such as Direct Preference Optimization (DPO)—have also been successfully applied to align LLMs with human preferences. This method has shown to be more stable, efficient, and even more effective than RLHF in certain settings, while also being simpler to implement and train [**?**]. On the other hand, as we have already seen Multi-Agent Reinforcement Learning extends traditional RL to environments involving multiple interacting agents, where each agent learns and adapts its behavior based not only on the environment, but also on the evolving strategies of other agents. This can often allow to more precisely model different real-world environments and applications.

One of the fastest-growing areas in Multi-Agent Reinforcement Learning is the coordination and cooperation of agents in strategic environments, where achieving goals requires not only individual competence but also the emergence of complex, collective behaviors. A key development in this direction is the StarCraft Multi-Agent Challenge (SMAC) [**?**], which has become a widely adopted benchmark. SMAC enables researchers to evaluate the performance of their algorithms in environments that demand both cooperative and competitive strategies under partial observability and complex dynamics. A particularly notable example is AlphaStar, a project developed by DeepMind, which combines MARL with supervised learning. AlphaStar demonstrated superhuman performance in the real-time strategy game StarCraft II [**?**, **?**], showcasing the remarkable potential of MARL in high-dimensional, dynamic, and adversarial settings.

A rapidly growing subfield is that of language-based agents—agents equipped with natural language capabilities that enable them to communicate, negotiate, and collaborate to solve complex tasks [**?**]. This paradigm opens the door to the emergence of communication protocols and social behaviours that may resemble how humans interact, coordinate, and resolve conflicts. This language-based model approach can achieve a set of different behaviours such as, (1) interact with humans to better understand user needs, personalize responses, and align with human values (2) interact with knowledge bases to enrich responses with factual and contextual information (3) interact with models and tools to solve complex tasks by delegating subtasks and simulating social behaviour and (4) interact with environments to ground language in perception and perform tasks requiring reasoning, planning, and decision-making. Recent research has explored how multiple agents, each powered by a Large Language Model [**?**], can achieve these sets of behaviours, and essentially how LLMs can be leveraged as a foundation to build AI agents. For instance, in the Generative Agents framework [**?**], LLM-based agents simulated believable human behaviour in a shared environment, displaying emergent properties such as memory, planning, and social interaction—all mediated through natural language.

Another promising application of MARL is in traffic signal control, where decentralized agents are employed to learn policies that reduce congestion, improve travel times, and adapt to dynamic traffic patterns. Unlike traditional rule-based or heuristic systems, MARL agents are capable of learning from experience, allowing them to continuously improve their performance and adjust to evolving traffic behaviours over time. Recent studies [**?**] have demonstrated that MARL-based strategies can achieve significant improvements in traffic flow and average delays compared to conventional adaptive control methods. By enabling coordination among neighboring intersections, MARL allows

the emergence of cooperative behaviors that enhance the overall efficiency of the transportation network.

In the realm of autonomous driving, MARL is used to model the complex interactions among vehicles, particularly in dense or unstructured environments such as intersections, roundabouts, or highways. In these contexts, each autonomous vehicle is treated as an intelligent agent that learns to navigate, merge, and avoid collisions while also anticipating and reacting to the behaviour of surrounding agents. Recent work [**?**] explored cooperative driving using MARL in mixed-traffic scenarios, where autonomous and human-driven vehicles coexist. Their framework showed that cooperative policies could improve safety and efficiency without requiring explicit communication.

The MARL applications discussed so far fall under a decentralized approach, where agents learn their individual policies based solely on their own experiences, without requiring any centralized coordination or control. However, alternative paradigms exist—most notably *Centralized Training with Decentralized Execution* (CTDE) [**?**], which enables centralized access to global information during training while preserving decentralized execution at inference time. An alternative version of this centralized approach is proposed via *Centralized Advising and Decentralized Pruning* (CADP) [**?**] that introduces an explicit communication channel that allows agents to request and receive advice from others, thereby facilitating a more centralized form of training. To ensure fully decentralized execution at test time, inter-agent communication is gradually pruned, ensuring that agents operate fully independently without any communication channels, thus preserving decentralized deployment. The proposed CADP framework demonstrates promising results on the StarCraft Multi-Agent Challenge (SMAC) benchmark, outperforming several state-of-the-art methods.

Another notable proposal is *Counterfactual Multi-Agent Policy Gradients* (COMA) [**?**], a CTDE method based on an actor-critic architecture [**?**] that employs a centralized critic and decentralized actors to optimize each agent's policy. COMA was also evaluated on the StarCraft Multi-Agent Challenge, where it outperformed other centralized baselines. The ultimate goal of this approach is to be applied to real-world scenarios, such as autonomous driving systems.

CHAPTER 3

A Supervising Centralized Approach to Multi-Agent Reinforcement Learning

In this chapter, we introduce a theoretical framework referred to as *Generic Multi-Agent Reinforcement Learning* (GMARL). As discussed in Chapter **??**, we employed Markov Decision Processes for Multiple Agents (MMDPs) and Stochastic Games (SGs) to formally characterize fully cooperative and cooperative settings, respectively. In what follows, we provide a formal definition of the GMARL framework, along with an intuitive explanation of its underlying principles. Additionally, we extend the framework to support environments with *partial observability*—that is, settings in which agents have limited access to the global state and must rely on local or incomplete observations. As will be evident in the experimental section, the environments we consider exhibit precisely these characteristics. Consequently, it is crucial to develop a framework that can operate effectively under partial observability, without assuming access to full state information.

Additionally, as it becomes evident through the GMARL definition, distributed approaches to Multi-Agent Reinforcement Learning typically require learning a separate policy π_i for each agent. While this allows for individualized behaviours, it also introduces significant challenges in terms of time and computational resources. To address these limitations, we propose a centralized framework that mitigates some of the inefficiencies of decentralized learning, while also tackling the scalability issues inherent to centralized methods. In particular, we introduce a meta-agent, referred to as the **supervisor** [**?**], which sequentially assigns actions to individual agents to promote cooperative behaviour.

Furthermore, we present a novel reward aggregation method that enables the supervisor to process a vector of rewards from all agents in the environment and convert it into a single scalar value. This scalar provides a meaningful measure of the overall team performance. We will discuss the benefits of this aggregation technique compared to simpler alternatives.

3.1 Generic Multi-Agent Reinforcement Learning

To begin with, let's start by intuitively defining how partial observability works. Typically, agents perceive their environment through a sensor or sensors that capture only a subset of all the information available in the environment. This information will, in most cases, be incomplete to make a fully informed decision. Generally speaking, par-

tial observability means the impossibility of total access to an environment state. We can identify two reasons for the partial observability of an agent:

- **Range of a sensor**. The range of a sensor refers to the maximum and minimum values that it can reliably detect or measure. For example, the range of a position sensor may not cover the whole room, but only an area close to the sensor, limited by certain parameters. In this case, partial observability of the state implies that some state variables of the environment state are inaccessible and therefore no value is read for such variables.
- **Noise of a sensor**. It refers to the noise induced by the sensor when measuring a variable. In this case, partial observability implies a sensor reading error, i.e. the state variable may have a possibly incorrect, distorted or biased value.

When the source of partial observability comes from the range of sensors, we end up with partial states; that is, states in which the state variables have correct values but some of the variable values are missing. When the source of partial observability comes from the noise of the sensors, we end up with variables with inaccurate values. The partial observability in the first case can be specified with a function that, for a given state s, it returns what is observed from s, i.e., a partial state or observation. In general, one knows the range of the problem sensors, and so we can know upfront the observations that will be captured with the sensors.

The second case is more complex because the failure or misreading of the sensors is something that is not known in advance. Consequently, the partial observability here is defined as a probabilistic distribution over the states, more particularly over the value domain of the variables that make up the state. This work centers on the first of these formulations, under the assumption of environments devoid of observational noise and stochasticity. Such settings correspond to multi-agent environments where agents operate under partial or limited observability. Accordingly, the definitions presented below must be adapted to reflect this particular constraint.

We begin by introducing *Partially Observable Markov Decision Processes* (POMDPs) [**?**], which generalize the intuition behind MDPs to situations where the agent does not have access to the full state of the environment and must instead rely on partial observations. As stated in Definition **??**, the underlying intuition of POMDPs is very similar to that of MDPs; however, at a given timestep t, the agent perceives a partial observation $o_t \in O$ instead of the full state s_t, through the observation function $\mathcal{F}$. This function takes the current environment state s_t as input and returns the observation the agent is able to perceive from it (e.g., through available sensors).

Definition 4 *A Partially Observable Markov Decision Process (POMDP) is a tuple* $\langle S, A, \mathcal{T}, \mathcal{R}, \mathcal{F}, O \rangle$ *where* S *is the state space,* A *is the finite sets of actions available to the agent,* $\mathcal{T} : S \times A \times S \to [0, 1]$ *is the state transition probability function,* $\mathcal{R} : S \times A \times S \to \mathbb{R}$ *is the reward function,* O *is the set of possible observations. At timestep* t *the agent perceives observations* $o_t \in O$ *according to the observation function* $\mathcal{F} : S \to O$.

It is important to note that, in our formulation, the observation function $\mathcal{F}$ is assumed to be deterministic, as opposed to a stochastic function of the form $\mathcal{F} : S \times O \to [0, 1]$. This assumption implies that, for a given state s_t, the agent's sensors will always return the same observation o_t. The stochastic formulation would instead account for situations where, for example, sensors introduce noise, leading to slightly different observations even for the same underlying state. As mentioned, we assume noise-free environments

in this work, and therefore we adopt the deterministic formulation and disregard the stochastic one. Another important aspect of this framework is that the agent's policy does not rely on the full state s to select an action, but rather on the observation available to the agent. This requires us to redefine the policy function as follows:

$$\pi : O \times A \rightarrow [0, 1] \tag{3.1}$$

Under this definition, we denote by $\pi(a \mid o)$ the probability that the agent selects action a given observation o. This policy is the object to be optimized during training, with the optimization objective being the maximization of the expected discounted return from the initial timestep, as previously defined in Equations **??** and **??** in Chapter **??**. After presenting some essential concepts related to partial observability, we proceed to generalize these definitions to the multi-agent case. We proceed with the formal definition of *Generic Multi-Agent Reinforcement Learning* (GMARL) as follows. This definition is loosely inspired by the formalism of *Decentralized Partially Observable Markov Decision Processes* (Dec-POMDPs) [**?**].

Definition 5 *A Generic Multi-Agent Reinforcement Learning Problem (GMARL) is a tuple* $\langle S, A_1, \ldots, A_m, \mathcal{T}, \mathcal{R}_i, \ldots, \mathcal{R}_m, \mathcal{F}_1, \ldots, \mathcal{F}_m, O_1, \ldots, O_m \rangle$ *where m is the number of agents, S is the state space, $A_i, i = 1, \ldots, m$ are the finite sets of actions available to the agents, yielding the joint action set $\mathbf{A} = A_1 \times \ldots A_m$, $\mathcal{T} : S \times \mathbf{A} \times S \rightarrow [0, 1]$ is the state transition probability function, $\mathcal{R}_i : S \times \mathbf{A} \times S \rightarrow \mathbb{R}$, $i = 1, \ldots, m$ are the reward functions of the agents, and $O_i, i = 1 \ldots, m$ are the individual observations of each agent that form the joint observation $\boldsymbol{O} = O_1 \times \cdots \times O_m$. Each agent, at timestep t perceives observations $o_t^i \in O_i$ according to the observation function $\mathcal{F}_i(s) : S \rightarrow O_i$, $i = 1, \ldots, m$.*

Intuitively, in a GMARL, at each timestep t, an agent perceives partial information about the environment. The observation of each agent is obtained by means of a certain observation function $\mathcal{F}_i$ such that $\mathcal{F}_i(s_t) = o_i^t$ being $o_i^t \in O_i$ the observation perceived by the i-th agent at time t from state s_t. Both the observation function $\mathcal{F}_i$ and the set of possible observations O_i are unique to each agent, which enables that even for the same state s_t agents perceive different observations of the environment (e.g. because they have different sensors to perceive it). Once the agent obtains the observation of its environment, it uses its policy π_i to choose, stochastically, which action to take. The policies π_i adopt the structure outlined in Equation **??**, with the global observation and action spaces O and A substituted by the agent-specific observation and action spaces O_i and A_i, respectively.

Once each agent has selected its respective action, the joint action $\mathbf{a}_t$ is formed. The environment then transitions to the next state s_{t+1} according to the stochastic transition function $\mathcal{T}$. Given the transition tuple $(s_t, \mathbf{a}_t, s_{t+1})$, each agent receives an individual reward determined by its reward function $\mathcal{R}_i$. This reward is then used to update and refine the agent's policy π_i. In Figure **??**, we show a diagram with three agents to visually illustrate how the agent-environment interaction works within the GMARL framework. As can be seen, agents perceive their environment through the function $\mathcal{F}_i$. Additionally, each agent receives a different reward after the joint action $\mathbf{a}_t$ is applied and the system transitions to state s_{t+1}.

Having presented the workings of GMARL, we now outline how it is capable of encompassing both fully cooperative and cooperative problems. On the one hand, as previously discussed, fully cooperative problems can be formalized through a Multi-Agent Markov Decision Process, characterized by agents receiving an identical reward at each timestep. To model such scenarios within GMARL, we impose the condition

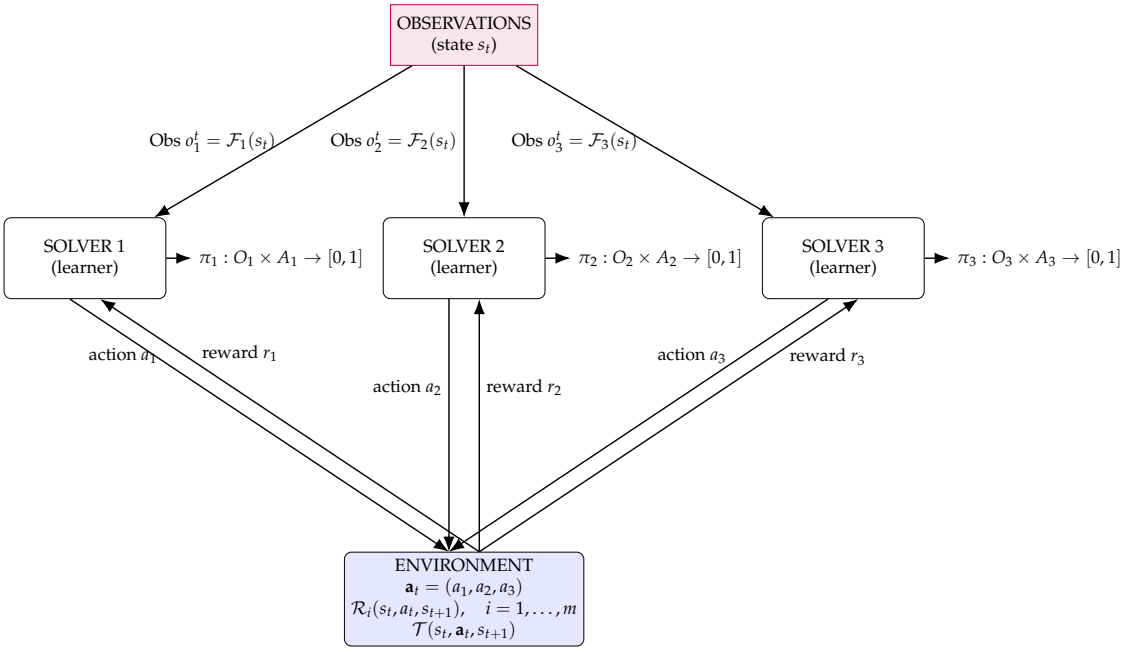


Figure 3.1: Agent-environment interaction loop for GMARL

$\mathcal{R}_i = \mathcal{R}_j$, $\forall i, j \in \{1, 2, \ldots, m\}$, meaning that all agents share the same reward function. This ensures that agents always perceive the same reward signal, thereby capturing the idea that they are working towards a common goal by maximizing this shared reward.

On the other hand, recall that cooperative problems are formalized as Stochastic Games, which are distinguished by agents having individual reward functions. These problems are naturally accommodated by the GMARL framework, without requiring any additional assumptions. By allowing for heterogeneous reward functions, GMARL enables the modeling of environments in which agents pursue partially aligned or individual objectives, while still requiring a significant degree of cooperation to act effectively and achive a shared goal.

3.2 Centralized Approach to MARL

As previously mentioned, and as we have also seen with the previous GMARL definition, distributed multi-agent approaches involve learning a separate policy π_i for each agent. This method leads to agents learning to make decisions and obtain outcomes independently. The main advantages of this approach lie in its scalability, resilience, and adaptability, as agents can operate with partial information and respond to local changes. However, due to its decentralized nature, this method may encounter issues related to global consistency and optimization. In contrast, centralized approaches rely on a single meta-agent that receives and processes information from all agents, assigns actions, and coordinates their behaviour. While this setup reduces the problem to learning a single policy instead of n separate ones, it may suffer from scalability limitations and other bottlenecks. To mitigate some of these issues, our approach involves reducing the policy search space from a Multi-Agent MDP to a Single-Agent MDP.

Intuitively, in a centralized multi-agent environment, at each timestep t, a joint action is generated as $\mathbf{a}_t = (a_1, \ldots, a_n)$, where n is the number of agents and $a_i \in A_i$ is the action taken by agent i. Executing this joint action results in a new state s_t and a reward vector $\mathbf{r}_t = (r_1, \ldots, r_n)$, containing the individual rewards received by each agent. Con-

sequently, in a centralized MARL setting, the overall action space becomes the Cartesian product of the individual action sets, i.e., $A = \prod_{i=1}^{n} A_i$. In most multi-agent environments, all of them usually share the same action space. This is because the task of coordinating them to undertake certain tasks usually involves agents with the same action capabilities. For this reason, we will adopt this assumption to improve the readability and comprehensibility of the following sections and, therefore, $A_i = A_j, \forall i, j \in \{1, 2, \ldots, n\}$.

In this centralized approaches, a single learning entity acts as an intermediary between the environment and the agents. However, learning over the joint action space quickly becomes impractical. For example, consider an environment with four agents Ag_1, Ag_2, Ag_3, Ag_4, each capable of executing ten actions from the set $A = \{a_1, a_2, \ldots, a_{10}\}$. The size of the joint action space in this case would be $|A|^4 = 10^4 = 10000$, which highlights the combinatorial explosion. Each joint action can be interpreted as a specific assignment of actions to agents. For instance, the joint action $\mathbf{a}_t = (a_2, a_1, a_3, a_2)$ indicates that agents Ag_1 through Ag_4 perform actions a_2, a_1, a_3 and a_2, respectively. These actions are executed simultaneously at timestep t, resulting in a reward vector as previously described. The core challenge with this approach is that the size of the joint action space grows exponentially with the number of agents, rendering it intractable for many Deep Reinforcement Learning algorithms.

To alleviate this combinatorial explosion, we introduce a meta-agent, called a **supervisor**, which builds joint actions sequentially. Following the previous example, this joint action $\mathbf{a}_t = (a_2, a_1, a_3, a_2)$ could be sequentially constructed by first assigning a_2 to Ag_1, secondly assigning a_1 to Ag_2, thirdly assigning a_3 to Ag_3 and lastly a_2 to Ag_4. There is no need to specify which agent the action is assigned to, as the process of assigning actions is done in a specified agent order. The supervisor's action set A' then becomes isomorphic to the individual action space A and so it contains only $|A'| = |A| = 10$ actions, namely *assign* a_1, *assign* a_2, *assign* a_3, *assign* a_4, *assign* a_5, *assign* a_6, *assign* a_7,*assign* a_8, *assign* a_9 and *assign* a_{10}.

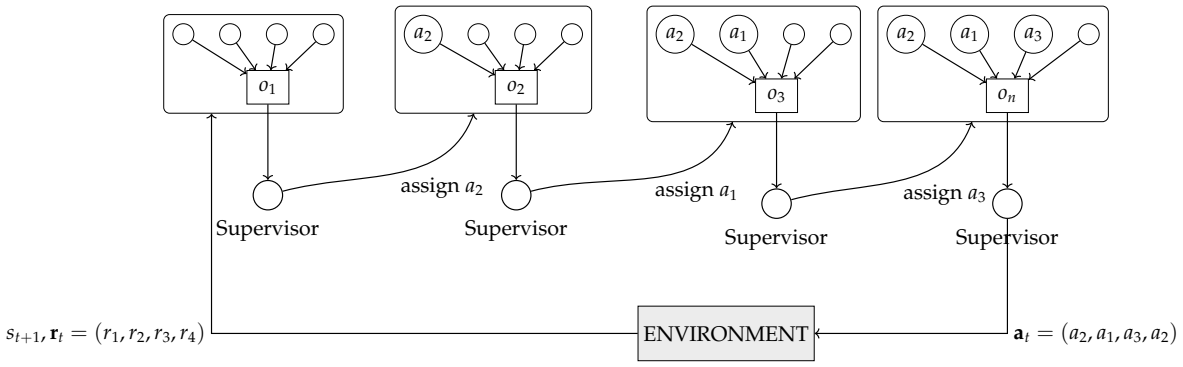


Figure 3.2: Joint action construction using supervisor meta-agent.

To better capture the intuition of how the supervisor meta-agent works, we show in Figure **??** a small diagram of how it iterates to construct a joint action. It should be noted that, as we have previously introduced, we are going to work with environments with partial observability, i.e., where agents have a limited ability to perceive their environment. Consequently, the supervising agent will not be able to perceive the global state of the environment in its completeness either, which will force it to rely on the individual observations of each agent.

As can be seen, the supervisor starts by perceiving o_1, i.e. the observation of the first agent. From this observation, it makes the decision to take the action *assign* a_2, i.e. to assign action a_2 to the first agent. This action is 'recorded' in a sort of history, which allows the supervising agent to know which actions it has assigned so far. It then

perceives the observation of the second agent and, using this observation and the history of actions already assigned, takes the action *assign* a_1. This process is repeated until, eventually, the supervisor has assigned actions to all the agents. Once the assignment is complete, the supervisor constructs the joint action $\mathbf{a}_t$ and launches its execution on the environment. This execution will result in a potentially different state than the previous one, s_{t+1}, as well as a rewards vector $\mathbf{r}_t$ containing the individual rewards of each agent for performing the assigned action in the environment.

Once this iteration concludes, the action assignments to the agents are reset and the entire process begins anew. It is important to highlight that the action assignment process for the agents is consistently performed in a fixed order. Once an ordering of the agents is defined, the supervisor adheres to this sequence throughout all iterations. Through this implementation, we manage to learn a single policy for the supervisor. The behaviour of this supervisor thus consists of the sequential construction of joint actions. It is worth mentioning that, through this abstraction, the supervisor does not need explicit knowledge of the agents' transition systems, since its sole purpose is to generate these joint actions and launch their execution on the environment. Having understood this centralized approach via the supervisor meta-agent, we proceed in the next section to present a more detailed and formal implementation of the process described here.

3.3 Construction of joint actions

In this section, we provide a formal description of the process for constructing joint actions through the supervisor meta-agent. This process can be understood as a compilation of a Generic Multi-Agent Reinforcement Learning Problem (GMARL) into a single-agent Partially Observable Markov Decision Process (POMDP). Given a GMARL $M = \langle S, A_1, \ldots, A_m, \mathcal{T}, \mathcal{R}_1, \ldots, \mathcal{R}_m, \mathcal{F}_1, \ldots, \mathcal{F}_m, O_1, \ldots, O_m \rangle$ that models a multi-agent environment where all agents share the same action set, i.e., $A_i = A_j$ for all $i, j \in \{1, \ldots, m\}$, which we will denote simply as A, we define a compilation process that transforms M into an abstract POMDP $M' = \langle S', A', \mathcal{T}', \mathcal{R}', \mathcal{F}', O' \rangle$.

- Observations $o' \in O'$ are defined as meta-observations of the form $o' = (o^i, L)$, where $o^i \in O_i$ is an observation from the i-th agent in the original GMARL M, and $L = (e_i)_{i=1}^m$ is a sequence representing the current action assignments. Each $e_i \in A \cup \{-\}$, where "$-$" indicates that no action has been assigned yet to agent i.
- The set A' of *meta-actions* is defined as $A' = \{assign\ a \mid a \in A\}$.
- The *meta-reward* function $\mathcal{R}'$ can be defined as follows:
  - If $o' = (o^{k+1}, (a_{i_1}, a_{i_2}, \ldots, a_{i_k}, -, \ldots, -))$, $o^{k+1} \in O_{k+1}$ and $a_{i_{k+1}} \in A$ then $\mathcal{R}'(o', assign\ a_{i_{k+1}})) = 0$. This means that, when the number of "$-$" in the sequence is more than one, then the reward should be 0, as the assignment process is not finished yet.
  - If $o' = (o^n, (a_{i_1}, a_{i_2}, \ldots, a_{i_{n-1}}, -))$, $o^n \in O_n$ and $a_{i_n} \in A$ then $\mathcal{R}'(o', assign\ a_{i_n})) = Aggregate((\mathcal{R}_i(s_t, (a_{i_1}, \ldots a_{i_{n-1}}, a_{i_n}), s_{t+1})_{i=1}^n)$. This means that, when only one agent is left to be assigned an action, we obtain a **scalar value of the reward by aggregating the rewards of each agent**. These rewards are obtained after executing the joint action on the environment. Further in this chapter we will discuss how this aggregation can be done.
- The transition function $\mathcal{T}'$ is defined as:

- If $o' = (o^{k+1}, (a_{i_1}, a_{i_2}, \ldots, a_{i_k}, -, \ldots, -))$, $o^{k+1} \in O_{k+1}$ and $a_{i_{k+1}} \in A$ then $\mathcal{T}'(o', assign\ a_{i_{k+1}}) = (\mathcal{F}_{k+2}(s_t), (a_{i_1}, a_{i_2}, \ldots, a_{i_k}, a_{i_{k+1}}, -, \ldots, -))$. This means that, when the number of "$-$" in the sequence is more than one, then the supervisor assigns the action to the next agent. The observation associated with the next meta-observation is derived by applying the observation function of the subsequent agent, $\mathcal{F}_{k+2}$, to the current environment state s_t.
- If $o' = (o^n, (a_{i_1}, a_{i_2}, \ldots, a_{i_{n-1}}, -))$, $o^n \in O_n$ and $a_{i_n} \in A$ then $\mathcal{T}'(o', assign\ a_{i_n})) = (\mathcal{F}_1(\mathcal{T}(s_t, (a_{i_1}, a_{i_2}, \ldots, a_{i_{n-1}}, a_{i_n}))), (-)_{i=1}^n)$. This means that the supervisor ends the action assignment, the joint action is launched for execution on the environment, and the joint action is reset to all "-". The next observation of the meta-observation, o', corresponds to that of agent 1, as executing the joint action in the environment resets the action selection process. Consequently, we use $\mathcal{F}_1$ to obtain the observation for the first agent.

Algorithm 3.1 Description of a step of the sequential construction of a joint action

Input: MARL environment E, meta-state (o^i, L_i), current environment state s_t, meta-action $(\text{assign } a)$
Output: Next meta-state (o^{i+1}, L_{i+1}), meta-reward r

1: **procedure** STEP($E, (o^i, L_i), s_t, (\text{assign } a)$)
2: **if** Count$(L_i, -) > 1$ **then**
3: /* If there are still actions to be assigned, assign them to the tuple */
4: $L_{i+1} \leftarrow L_i$
5: $k \leftarrow$ GetFirstPosition$(L_i, -)$
6: $L_{i+1}[k] \leftarrow a$
7: $o^{i+1} \leftarrow \mathcal{F}_{i+1}(s_t)$
8: $r \leftarrow 0$
9: **else**
10: /* If next action is the last one, add it and compute next step using the original environment */
11: $k \leftarrow$ GetLastPosition$(L_i, -)$
12: $L_i[k] \leftarrow a$
13: $(s_{t+1}, \mathbf{r}_t) \leftarrow$ Execute(E, s_t, L_i)
14: $r \leftarrow$ Aggregate$(\mathbf{r}_t)$
15: $L_{i+1} \leftarrow (-, \ldots, -)$
16: /* We return the observation of the first agent */
17: $o^{i+1} \leftarrow \mathcal{F}_1(s_{t+1})$
18: **end if**
19: **return** $(o^{i+1}, L_{i+1}), r$
20: **end procedure**

To better clarify this process, we show in the Algorithm **??** a pseudocode that implements the construction of a joint action by the supervisor agent. This algorithm is invoked each time the supervisor takes a step, i.e. assigns an action. The function takes as input a certain multi-agent environment E, a meta-state (o^i, L_i), the current state of the environment s_t and the meta-action generated by the supervisor (*assign a*) and returns the next meta-state (o^{i+1}, L_{i+1}) and the perceived meta-reward r.

The process begins by counting the number of occurrences of the symbol "-" in L_i (line 2). If there is more than one occurrence, the algorithm proceeds with lines 3 to 8, where the following steps are performed: L_i is copied into L_{i+1}; the first position containing "-" is identified; and the received action a (from the meta-action *assign a*) is assigned to that

position (lines 4 to 6). Then, the observation of the next agent, o^{i+1}, is obtained using its corresponding observation function $\mathcal{F}_{i+1}$, and a reward of 0 is returned (lines 7-8), since no real interaction with the environment has taken place yet.

On the other hand, if the number of occurrences of the symbol "-" in L_i is equal to 1, we must start applying actions to the environment, which corresponds to executing lines 11 to 17. Specifically, the action a is assigned to the last remaining position in L_i, and the environment E is stepped using the complete joint action specified in L_i (lines 11 to 13). This produces a vector of rewards $\mathbf{r}_t$, which is then aggregated into a scalar reward r using the *Aggregate* function, as detailed in the next section (line 14). Next, L_{i+1} is reset by initializing all its positions with the symbol "-" (line 15). Due to the iterative nature of the process, we assign o^{i+1} to be the observation of Agent 1, obtained by applying its corresponding observation function $\mathcal{F}_1$ to the new environment state s_{t+1} (line 17). Finally, the algorithm returns the new meta-state (o^{i+1}, L_{i+1}) along with the reward signal r computed in this step (line 19).

One of the main advantages of this implementation is its full compatibility with single-agent reinforcement learning algorithms. Specifically, by introducing a *supervisor meta-agent* that learns to assign actions to the individual agents in the environment, we abstract the multi-agent learning problem, typically requiring the training of n separate policies, into the learning of a single policy. This reformulation enables the direct application of standard single-agent algorithms, such as Proximal Policy Optimization (PPO) [**?**], to the multi-agent setting.

3.4 Reward Aggregation Methods

As defined in Chapter **??**, the reward function in a Markov Decision Process (MDP) for the single-agent case takes a tuple (s_t, a_t, s_{t+1}) and returns a scalar reward $r \in \mathbb{R}$. For this reason, single-agent RL algorithms are generally designed to optimize scalar rewards and are not inherently compatible with reward vectors. Consequently, the possibility arises of exploring different aggregation methods that, given a reward vector, compute an aggregation to obtain a single scalar value. As defined in Algorithm **??**, once the actions have been assigned to all agents, the supervising meta-agent must calculate an aggregation of the vector of rewards $\mathbf{r}_t$, obtained by applying all the actions on the environment at certain timestep t. This reward vector contains the individual rewards of each of the agents, and the aggregation method must return a scalar value from them. This aggregation method must be carefully designed to prevent the collapse of distinct reward vectors into an identical scalar value. We now proceed to examine several alternative aggregation strategies.

3.4.1. The Sum aggregation

A first approximation to solve this problem would consist of simply calculating a sum of all the elements of the vector as follows.

$$SumAggregation(\mathbf{r}) = \sum_{i=1}^{n} r_i \tag{3.2}$$

This form of aggregation, although simple, can be robust and convenient when working with fully cooperative problems. Recall that fully cooperative problems are those in which there is a single reward function and, therefore, the reward vector will always contain the same value, repeated as many times as there are agents in the environment.

Also recall that this type of problem can be modeled within our theoretical framework, GMARL, by imposing the condition $\mathcal{R}_i = \mathcal{R}_j,\ \forall i, j \in \{1, \ldots, m\}$; that is, by assuming that all agents share the same reward function. In these cases, aggregation by means of a sum allows us to obtain a scalar that, the higher its value, the better the performance of the agents.

To reinforce this intuition, let's suppose that we have an environment with four agents in which the maximum reward value they can obtain is 10 and the minimum value is -10. If for a certain execution of the actions assigned by the supervisor, we obtain a vector $\mathbf{r} = (-5, -5, -5, -5)$ then the aggregation would be $SumAggregation(\mathbf{r}) = -20$. On the other hand, if the agent launches the executions and we obtain a vector $\mathbf{r}' = (8, 8, 8, 8)$ then the aggregation would be $SumAggregation(\mathbf{r}') = 32$. Through this example, we try to capture the intuition that, if the rewards are equal for all agents, then the sum is sufficiently informative with respect to the overall performance, i.e. whether the agents are approaching the overall goal they share.

However, in cooperative problems where agents may perceive different reward signals, this form of aggregation can present certain issues. In these cases, agents will perceive a reward proportional to how well they perform. Therefore, it may be common for the supervising meta-agent to receive vectors with very disparate values. To clarify this idea, let's look at it again with an example. Suppose again that we have four agents in our environment, and that they can perceive different rewards. After executing the actions assigned by the supervisor, we obtain the following reward vector $\mathbf{r} = (-10, 10, 10, 10)$. In this case, if we calculate the aggregation by summation, we would obtain the following value $SumAggregation(\mathbf{r}) = 30$. On the other hand, suppose that after executing another joint action, the reward vector we now obtain is $\mathbf{r}' = (7, 8, 7, 8)$. If we calculate the aggregation by summation we would then obtain $SumAggregation(\mathbf{r}') = 30$.

Although the two scalar reward signals are identical, it is evident that the agents' behaviours differ significantly in each scenario. Assuming a minimum reward of -10 and a maximum of 10, in the first case, the first agent performs considerably worse than the other three agents, who achieve optimal performance. In contrast, in the second case, even though the scalar reward is also 30, the agents receive individual rewards around 7 or 8, which can be considered a more balanced performance of every agent. This example highlights that, in certain situations, summing rewards as an aggregation method can obscure the fact that some agents are underperforming significantly compared to others. Over time, this may cause stagnation in overall performance or lead to training outcomes where the apparent improvement is driven only by a subset of agents efficiently approaching their goals, while others lag far behind. Considering these issues, it becomes necessary to implement an aggregation method that accounts for performance imbalances among agents—specifically, one that produces more informative values whenever one or more agents exhibit substantially poorer performance than the rest. To achieve these results, we propose an aggregation of the reward vector using a *Softmin + Mean* approach.

3.4.2. The Softmin + Mean function

The Softmin + Mean approach arises from combining the usual mean with the softmin function [**?**] weighted by an alpha α parameter, as can be seen in Equation **??**. On the one hand, the *Mean* function is the usual mean of a vector, that can be seen in Equation **??**. On the other hand, the softmin function (Equation **??**) is a smoothed variant of the minimum, designed to differentially penalise the lowest values within a set. Unlike the classical minimum, which is extremely sensitive to negative outliers, the softmin uses an exponential transformation to weight the elements according to their value: the lower

a reward, the greater its weight in the result. Mathematically, it is defined as the negative of the logarithm of the exponential average of the negative values. This formulation smoothes the real minimum and provides a continuous and differentiable measure, useful in environments where it is desirable to prevent a single outlier from dominating the aggregation, while still strongly penalising the presence of underperformers in the evaluated group.

$$SoftminMean(\mathbf{r}) = \alpha \cdot Softmin(\mathbf{r}) + (1 - \alpha) \cdot Mean(\mathbf{r}) \tag{3.3}$$

where:

$$Softmin(\mathbf{r}) = -log(\sum_{i=1}^{n} exp(-r_i)) \tag{3.4}$$

$$Mean(\mathbf{r}) = \frac{1}{n} \sum_{i=1}^{n} r_i \tag{3.5}$$

Let us examine the Softmin function behaviour with an example, as illustrated in Figure **??**. Consider an environment with four agents, where the rewards received by the last three agents are fixed at 5, 6, and 5, respectively. We allow the reward of the first agent, denoted by x, to vary uniformly over the interval [0, 10], in order to examine how the Softmin function responds to changes in this parameter. As the plot shows, the value of the Softmin function increases monotonically with x, but at a decreasing rate. When x is small (i.e, significantly lower than the other three values in the vector), the Softmin function heavily prioritizes it, resulting in a much lower output. In this case, the term $\exp(-x)$ in Equation **??** dominates the sum inside the logarithm. However, as x increases and becomes equal or greater than the other fixed rewards, its influence on the exponential sum diminishes. The function value therefore becomes less sensitive to further increases in x, and asymptotically approaches the soft minimum of the remaining fixed values. In summary, the purpose of the Softmin function is to strongly penalize reward vectors in which one or more agents exhibit significantly lower performance compared to the rest.

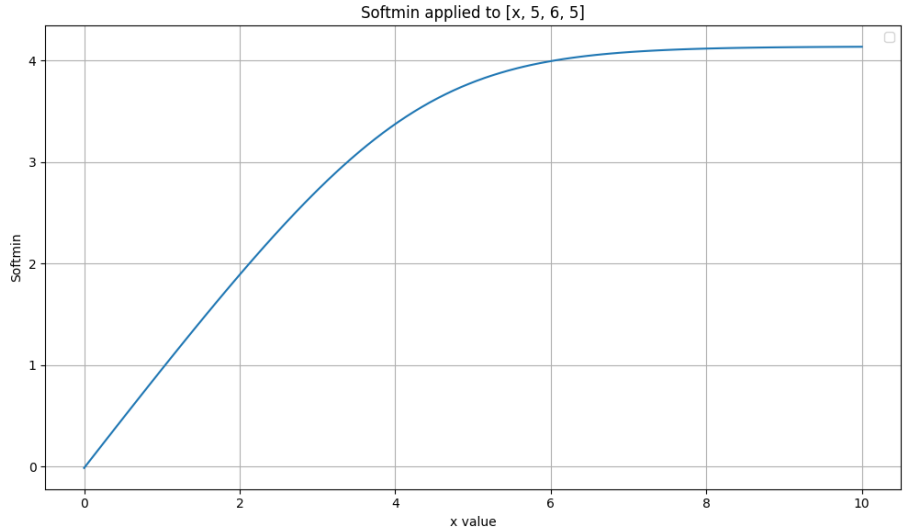


Figure 3.3: Softmin behaviour for reward vector $(x, 5, 6, 5)$

We now examine the results obtained when combining the $Softmin$ and $Mean$. The intuition behind combining these two functions is that, while we want to introduce a

penalty in the reward when agents exhibit unbalanced performance, we also need to complement this metric with a value that can reflect the overall performance of the agents. In other words, if we only considered the minimum reward, we could consistently observe low values even though the rest of the agents might be significantly improving their performance over time during training. Put differently, using only $Softmin$ as a reward estimator can be an overly pessimistic way of reflecting the agents' performance. For this reason, we also incorporate the $Mean$ function, which, despite one (or several) agents performing poorly, can still capture improvements in the performance of the others.

For example, if we initially have a reward vector $\mathbf{r} = (-5, 5, 5, 5)$ and later obtain $\mathbf{r}' = (-5, 10, 10, 10)$, then $Mean(\mathbf{r}) = 2.5$ and $Mean(\mathbf{r}') = 6.25$, meaning that we observe an increase in the scalar value when at least a subset of the agents improves their performance. If we were to use only $Softmin$, we would obtain values close to -5, which may serve as a pessimistic lower bound and potentially lead to stagnation during training.

The parameter $\alpha \in [0, 1]$ is included to weight the contribution of each function, allowing us to produce reward estimates that emphasize either outlier agents or the overall performance of the group. However, as previously justified, our goal with this approach is to obtain a scalar value that strikes a balanced compromise between penalizing outliers and reflecting the overall performance of the agents. If we set $\alpha = 0.0$, then our $SoftminMean$ function becomes equivalent to $Mean$, resulting in overly optimistic values. This is because $Mean$ can obscure the presence of outliers in agent performance. On the other hand, if $\alpha = 1.0$, then $SoftminMean$ becomes purely $Softmin$, which may lead to overly pessimistic reward estimates. Therefore, for our experiments, we choose $\alpha = 0.5$ as the weighting value to equally take into account the contribution of both estimators. In Figure **??**, we show the estimated value returned by $SoftminMean$ for different reward vectors when $\alpha = 0.5$.

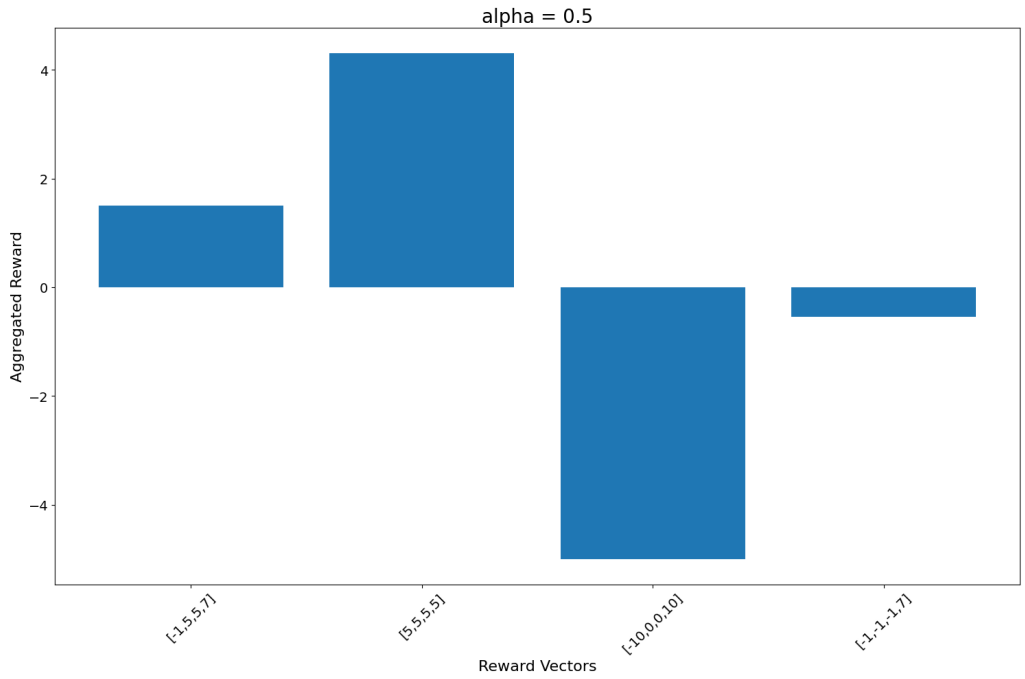


Figure 3.4: $SoftminMean$ behaviour for different reward vectors

Let us examine each of the aggregated reward results:

- $\mathbf{r} = (-1, 5, 5, 7)$: In this case, there is a clear outlier agent with significantly lower performance compared to the others. We observe that $SoftminMean$ returns an aggregated reward close to 2. This shows that the $Softmin$ component strongly

penalizes the presence of the outlier, but does not return a negative value, which would indicate an excessive emphasis on that single agent.

- $\mathbf{r} = (5, 5, 5, 5)$: Here, all agents exhibit completely homogeneous performance. As expected, $SoftminMean$ returns a value close to 5, correctly reflecting the balanced and consistent performance across agents without any penalty.

- $\mathbf{r} = (-10, 0, 0, 10)$: In this case, we observe an intermediate scenario: one agent performs much better than the others, two agents have moderate performance, and one performs significantly worse. $SoftminMean$ returns a value close to -5, indicating a substantial imbalance among the agents, with one of them dragging down the overall reward estimate.

- $\mathbf{r} = (-1, -1, -1, 7)$: This is a case in which one agent outperforms the others, who all have low performance. $SoftminMean$ returns a value slightly below 0, rather than exactly -1, indicating that the general performance is poor, but there is still a positive contribution from the outperforming agent.

3.4.3. Scalar Reward Normalization

We additionally propose a normalization of the reward values to the interval $[0, 1]$. Recall that in Algorithm **??**, whenever the supervisor meta-agent assigns an action but there are still actions left to be assigned, it returns a reward of 0. However, in our experimental phase, we work with environments that only use negative rewards, where the maximum achievable reward is 0. To prevent the reward associated with assigning a single action from being interpreted as the maximum possible reward in these settings, we propose a normalization scheme such that the reward for assigning actions remains 0, while the aggregation of the reward vectors always yields a result within the $[0, 1]$ range. This normalization is carried out on both Sum and Softmin + Mean aggregation methods.

To normalize the aggregated reward into the $[0, 1]$ interval, we rely on a dynamic estimate of the minimum and maximum reward values observed during training. Since the true bounds of the reward distribution are typically unknown a priori, we maintain an *exponential moving average (EMA)* of the minimum and maximum rewards encountered so far. More precisely, whenever a new reward value is computed, we update the minimum and maximum statistics using the following rule:

- If the new reward is lower than the current minimum, we update the minimum using an EMA:
$$\min_r \leftarrow (1 - \alpha_{\text{ema}}) \cdot \min_r + \alpha_{\text{ema}} \cdot reward \tag{3.6}$$

- Analogously, if the new reward is greater than the current maximum, we update the maximum as:
$$\max_r \leftarrow (1 - \alpha_{\text{ema}}) \cdot \max_r + \alpha_{\text{ema}} \cdot reward \tag{3.7}$$

Where *reward* refers to the aggregated reward value that we calculate via the $SoftminMean$ or $SumAggregation$ function, and α_{ema} is a real-valued parameter in the interval $[0, 1]$. This approach provides a smooth and adaptive estimate of the reward range, avoiding abrupt changes due to outliers or noise. By gradually updating the min/max values, we ensure that the normalization remains stable across training iterations. A small value of α_{ema} results in slower updates (more stability), while a larger value increases responsiveness to recent reward changes. The normalization expression used is the one that can

be seen in Equation **??**. To ensure numerical stability, a small constant ε is added to the denominator when computing the normalized reward, preventing division by zero.

$$\text{normalized reward} = \frac{\text{reward} - \min_r}{\max_r - \min_r + \varepsilon} \tag{3.8}$$

In cases where the maximum and minimum reward values are known in advance, they can be directly substituted into Equation **??**. Consequently, there is no need to rely on an EMA to progressively estimate these values over the course of training. Having thoroughly examined the centralized approach using the meta-agent supervisor, along with the reward aggregation methods we plan to use, we now proceed in the next chapter to describe in detail the Python implementation of this meta-agent. We also provide an overview of the PettingZoo API environments that will be used for our tests and experiments.

CHAPTER 4
Environment configuration

In this chapter, we describe the implementation of the supervisor meta-agent using Python as the programming language. We present how this agent is abstracted as an *OpenAI Gymnasium* environment, acting as a wrapper over environments from the PettingZoo library. We provide a detailed explanation of how the meta-agent performs key tasks such as aggregating reward vectors, maintaining the meta-states introduced in the previous chapter, and executing joint actions across each environment. Additionally, we explore how the meta-agent handles environments where agent observations are provided as images, and how it processes such visual inputs effectively.

Finally, we review the selected PettingZoo environments used in our experiments. We briefly describe each environment and propose a classification based on whether the underlying task is cooperative or either fully cooperative. This classification will later serve as the basis for selecting appropriate reward aggregation strategies in Chapter 6.

4.1 Supervisor meta-agent implementation

We begin with an in-depth review of the implementation of the supervisor meta-agent, using Python as the programming language of choice. As discussed in the previous chapter, this meta-agent abstracts a GMARL—an inherently multi-agent process—into a POMDP, that is, a single-agent process. Therefore, intuitively, our implementation should function as a single-agent interface that internally wraps around a multi-agent interface. With this intuition in mind, we begin by implementing a class called `SupervisorWrapper`, which inherits from `gym.Env`. In other words, this supervisor will behave like a typical *Gymnasium* environment from *OpenAI*. Before proceeding with the explanation, it is useful to briefly review the internal workings of a typical Gymnasium environment, as PettingZoo environments operate in a very similar manner.

4.1.1. Inner Workings of a Gymnasium Environment

The basic functionality of any Gymnasium environment, according to to the Gymnasium Documentation [**?**], requires the implementation of the `step` and `reset` methods, as well as the definition of the `action_space` and `observation_space` attributes. On the one hand, the `step` method computes a single timestep in the agent-environment interaction loop, as described in Chapter **??**. This method receives an action `action` as input, retrieves the agent's observation, executes the action in the environment, transitions to the next state, and finally computes the reward associated with the action. This method is essential for defining and implementing the environment's dynamics. On the other hand, the `reset` method simply resets the environment to its initial state, from which the simu-

lation begins. Typically, Gymnasium environments run for a fixed number of timesteps. Once this limit is reached, the `reset` method is called to reinitialize the environment's state, allowing the simulation to restart.

The `action_space` attribute defines the nature of the agent's actions. In general, actions can be either discrete or continuous. For discrete actions, the space is represented by a `Discrete(n)` object, indicating that the agent can choose from `n` distinct actions (e.g., moving in one of several directions on a grid). Therefore, an action would be represented as an integer between 0 and `n-1`. For continuous actions, the space is represented by a `Box(low, high, shape)` object, which specifies that the agent can take actions represented by real-valued vectors within the range `[low, high]` and with a given `shape`. As an example, consider an agent operating in a two-dimensional environment where it can apply a continuous acceleration vector. This vector might take values in the range `[-1, 1]` for each axis, representing acceleration in the positive or negative direction. In this case, the action space would be defined as `Box(-1, 1, (2,))`.

Finally, the agent's `observation_space` is also typically represented as a `Box(low, high , shape)` object, similar to continuous action spaces. The intuition behind this definition is that an agent's observation is a tensor of dimensionality `shape`, containing information about the subset of environment variables that the agent can perceive. For example, in a classic environment like Pong, an agent's observation could be a 4-dimensional vector with values ranging from -1 to 1. The first two entries might represent the position of the ball, while the remaining two entries correspond to its velocity vector.

These are the essential components of a Gymnasium environment. Since PettingZoo is a multi-agent API inspired by Gymnasium, many of its implementations and underlying concepts are reused. In PettingZoo environments, where there may be n agents, each agent has its own `action_space` and `observation_space`. This design provides enough flexibility for each agent to perform distinct actions and perceive different aspects of the environment. The `reset` method in PettingZoo works in the same way as in Gymnasium, reinitializing the environment to its initial state. The `step` method, however, executes the `action` provided for a specific agent. Internally, PettingZoo environments use a form of counter or iterator to keep track of which agent is currently expected to act. In other words, PettingZoo enforces a predefined agent ordering that dictates the sequence in which agents take their turns.

4.1.2. Supervisor Environment as a PettingZoo Wrapper

Now that we have a clearer understanding of the internal workings of a Gymnasium environment, we will proceed with a more in-depth review of how the meta-agent supervisor is implemented. As we have seen, to implement it as a Gymnasium environment, it is necessary to define a set of essential methods and attributes that govern its behavior. Furthermore, since it functions as a wrapper around PettingZoo environments, when instantiating an object of the `SupervisorWrapper` class, we must pass it a fully initialized and parameterized PettingZoo environment. Let's now take a closer look at each of these methods and attributes.

- `__init__`: During the initialization of the `SupervisorWrapper`, we receive a previously instantiated PettingZoo environment. From this environment, we initialize the `action_space` and `observation_space` variables. In our specific setting, where **all agents share the same observation capabilities**, we can initialize `observation_space` directly from any agent's observation space. Similarly, since the **action space is also homogeneous across agents**, we can follow the same approach for `action_space`. However, it is worth noting that in one of our environments, agents may

have *disjoint action spaces*. This requires some adjustments, which we will describe in Subsection **??**. Additionally, we initialize a list called `joint_action`, which will store the joint action vector $\mathbf{a}_t$ that the supervisor must construct at each step and then apply to the underlying PettingZoo environment. We also keep the number of agents in a variable `num_agents` to know if the joint action has been completely constructed (since, as a reminder, the joint action contains as many individual actions as there are agents in the environment).

- `step`: In this method, we need to implement Algorithm **??**, introduced in Chapter **??**. Recall that this method receives an `action` as input, which is first appended to the `joint_action` list. Then, we check whether the joint action has been fully constructed by comparing its length with the `num_agents` variable. If the joint action is not yet complete, the method returns a reward of 0 and advances to the next step. Otherwise, we iterate over the list and execute the individual actions on the underlying PettingZoo environment. For each executed action, we collect the corresponding reward and store it in a `rewards` list. Once all actions have been executed, the supervisor must **aggregate the `rewards` vector into a single scalar value**. This aggregation can be performed using either the $SumAggregation$ method (Equation **??**) or the $Softmin + Mean$ method (Equation **??**), depending on the configuration specified during the supervisor's initialization.

- `reset`: This method simply calls the `reset` method of the underlying PettingZoo environment and reinitializes internal variables such as `joint_action`.

It is also important to recall that, as discussed previously, the supervisor agent must maintain a "meta-observation" $o' = (o^i, L_i)$, where o^i is the observation of the i-th agent and L_i is a sequence representing the current action assignments for all agents. To translate this meta-observation concept into our implementation, the supervisor will, at each step, begin by perceiving the observation of the current agent, which will **always take the form of a one-dimensional vector**. To this vector, we concatenate the actions that have already been assigned—stored in the `joint_action` list.

To make this explanation more concrete, let's consider an example. Recalling the Pong environment introduced in the previous subsection, which involves two agents, the meta-observation vector in this case consists of a 4-dimensional vector (representing the position and velocity of the ball), to which we concatenate a 2-element vector (one element per agent) containing the currently assigned actions. For instance, the second agent might observe the vector $(30, 40, -0.5, 0.5)$, to which we append $(1, 0)$. This appended vector indicates that the first agent has been assigned discrete action 1, and the second agent—represented by the 0—has not yet been assigned any action.

4.1.3. Working with disjoint action spaces

As mentioned in the previous subsection, one of our environments involves disjoint action spaces. Specifically, we have two agents, one with an action space of `Discrete(3)` and the other with `Discrete(5)`. As we also noted in Subsection **??**, Gymnasium environments are designed for the single-agent case and therefore operate with a single, unified action space. Consequently, we must devise a strategy to handle these disjoint action spaces in a compatible manner.

To address this, we define a new action space $A = \Pi_{i=1}^{2} A_i$, that is, the Cartesian product of the individual action spaces of both agents. This product space can be abstracted as a single `Discrete(15)` action space, where each action index represents a unique pair

of agent actions. For example, action index 1 would correspond to the tuple (0, 1), meaning the first agent takes action 0 and the second takes action 1. This design allows us to capture all possible combinations of agent actions within a unified action space representation.

4.1.4. Optimization of the supervisor meta-agent

Once the meta-agent supervisor has been implemented, a policy optimization algorithm is needed to train the policy responsible for assigning actions to the agents in the underlying PettingZoo environment. For this purpose, we selected the *Proximal Policy Optimization* (PPO) algorithm [**?**]. PPO follows an Actor-Critic architecture, where the Actor proposes actions based on the current policy, and the Critic evaluates them by estimating a value function. These components are typically implemented as two separate neural networks, or a single network with distinct output heads. The Critic's feedback guides the Actor in improving its policy over time, enabling stable and efficient learning.

Although existing implementations of PPO are available, such as those provided by *Stable-Baselines3* [**?**], in our specific case, we opted for a custom implementation using the *PyTorch* framework [**?**]. This decision was based primarily on PyTorch's greater flexibility, which allows for fine-grained customization of the training pipeline compared to more rigid libraries like Stable-Baselines. Furthermore, PyTorch enables seamless loading of tensors and models onto CUDA-compatible devices when available, significantly accelerating both training and inference.

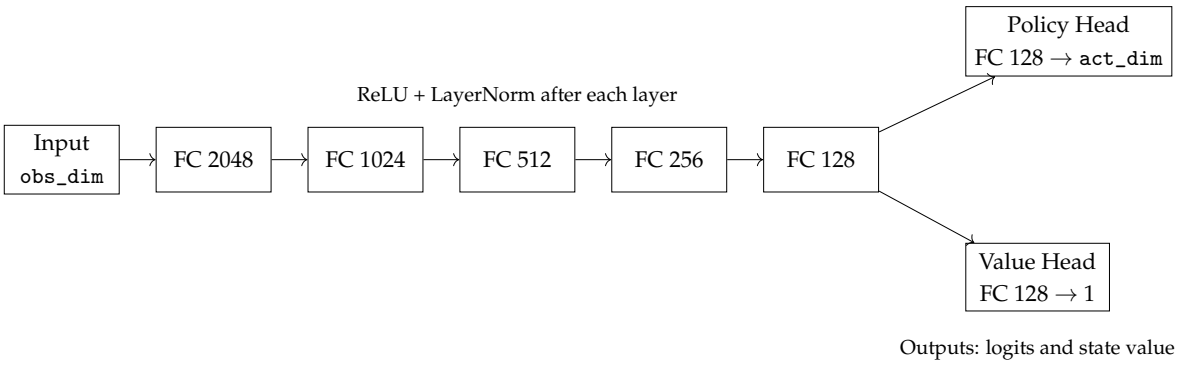


Figure 4.1: Neural network architecture used for PPO: shared MLP encoder with policy and value heads.

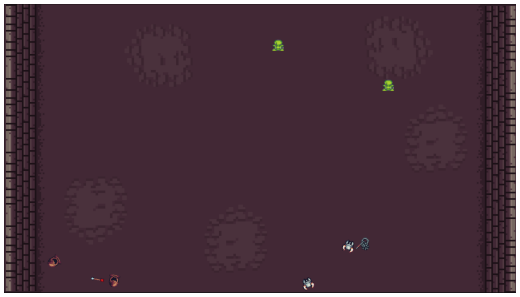
For our PPO implementation, we designed a deep neural network architecture with a shared feature extractor followed by separate heads for the policy and value functions. The shared network consists of five fully connected layers with hidden sizes of 2048, 1024, 512, 256, and 128 units, respectively. Each layer is followed by a ReLU activation and a Layer Normalization operation, which helps stabilize training by normalizing the outputs across the batch. This shared representation is then passed to two separate output heads. On the one hand, the policy head, which outputs logits over the action space using a fully connected layer of size $128 \times$ `act_dim` being `act_dim` the cardinality of the action space. On the other hand the value head, which produces a scalar estimation of the state value through a fully connected layer of size 128×1. A visual representation of this architecture can be seen in Figure **??**.

4.2 PettingZoo Environments

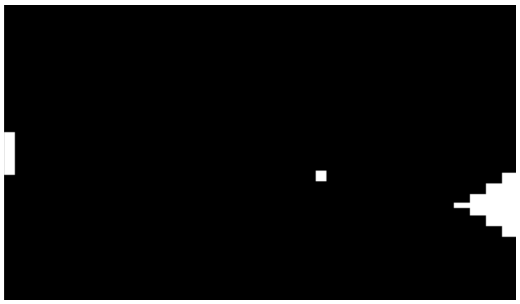
In this section, we provide a more detailed explanation of the PettingZoo environments used in our experimental phase. We chose to work with this API because, as stated in the official OpenAI documentation, it is the only multi-agent reinforcement learning library actively maintained by the Farama Foundation. Specifically, we will use environments from the *Butterfly*, *Multi-Particle Environments* (MPE) [**?**, **?**], and *Stanford Intelligent Systems Laboratory* (SISL) [**?**] categories. Furthermore, we will classify these environments as either fully cooperative or cooperative, based on the nature of the reward functions and the agents' objectives. Finally, we include a brief description of the observation *vectorization* process for environments based on image inputs.

4.2.1. Butterfly Environments

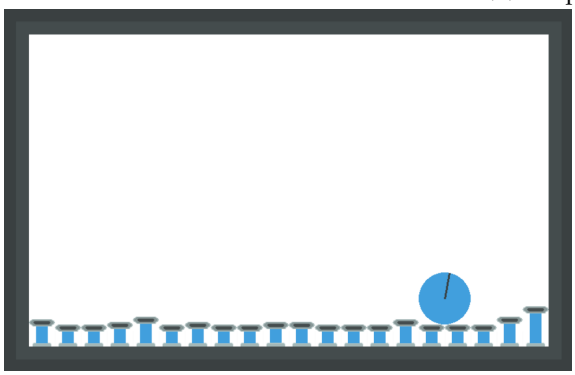
Butterfly environments are challenging scenarios created by Farama, using Pygame with visual Atari spaces. All environments require a high degree of cooperation and require learning of emergent behaviours to achieve an optimal policy. Accordingly, our experiments will focus on the **Cooperative Pong**, **Knights, Archers & Zombies**, and **Pistonball** environments, which are briefly described below. Representative visual examples of each environment are provided in Figure **??**.



(a) Knights Archers Zombies



(b) Cooperative Pong



(c) Pistonball

Figure 4.2: Example of each of the Butterfly environments

Knights, Archers & Zombies is an environment in which we control agents whose role can be either archer or knight. Each agent can rotate clockwise or counter-clockwise and move forward or backward; and are initially positioned at the bottom border of the environment. With this set of actions, the agents can move in an environment in which zombies walk from the top border to the bottom border following random paths. The agents objective is to defeat the zombies before any one of them gets to the bottom of the environment. The game ends when all agents die (by colliding with a zombie) or some

zombie gets to the bottom part of the environment. Knights can attack by swinging their mace in front of their current heading direction and archers can fire an arrow in the direction of their heading. As we saw in Chapter **??**, each agent receives a positive reward whenever it successfully eliminates a zombie so, although all agents share the same objective, they receive individual reward signals based on their personal contributions. We therefore classify Knights, Archers & Zombies as a **cooperative** problem.

Cooperative Pong is a simple pong game where the objective is to keep the ball in play for as long as possible. In this environment, there are two agents (paddles): one moves along the left edge of the screen, and the other along the right edge. At the start of each game, a ball spawns at the center of the screen and begins to move randomly either to the right or left. The agents must move their paddles to collide with the ball, which elastically rebounds upon collision, changing its direction. The game ends when the ball goes out of bounds through either the left or right edge of the screen. Each agent receives a reward proportional to `max_reward / max_cycles`, where `max_reward` and `max_cycles` are user-configurable parameters. This reward is granted at every timestep as long as the ball remains in play. If the ball crosses the screen limits, the agents receive a penalty proportional to `penalty_score`, another adjustable parameter. Therefore, this environment can be modeled using a single shared reward function among the two agents. Therefore, Cooperative Pong can be classified as a **fully cooperative** problem. It is worth noting that this environment is image-based, meaning that agents receive observations in the form of RGB images. Specifically, each observation corresponds to a full visual snapshot of the environment, as shown in Figure **??**. Therefore, in this environment, observations of the agents take the form of RGB images and not one-dimensional vectors. which correspond to the observation format expected by the supervisor meta-agent. Later in this chapter we will explain in greater detail how we deal with environments with these kind of observations.

Pistonball is a physics-based cooperative game where the objective is to move a ball to the left edge of the screen by activating a series of vertically moving pistons. Each agent controls one of these pistons and can observe the two adjacent pistons as well as whatever is directly above it (either the ball or empty space). Each agent can choose to move upward, downward, or remain stationary. The learning process is driven by a shared reward function among all agents. This reward is provided to each agent based on how much the ball moved left in the last time-step (moving right results in a negative reward). The distance component is the percentage of the initial total distance (i.e. at game-start) to the left-wall traveled in the past timestep. Additionally, a time penalty is introduced, reducing the overall reward based on the time taken to move the ball to its target position. Therefore, Pistonball can be classified as a **fully cooperative** problem. In this case, the agents' observations also take the form of RGB images. Specifically, each agent has a visual field that includes its two adjacent pistons (or the wall, if located at the edge) and the space above them, where the ball may potentially appear.

4.2.2. MPE Environments

Multi Particle Environments (MPE) are a set of environments where particle-like agents can move, communicate, see each other, push each other and interact with certain fixed landmarks. Our experiments will focus on the **Simple Reference**, **Simple Speaker Listener** and **Simple Spread** environments, which are briefly described below.

Simple Reference is an environment with two agents and three landmarks. Each agent must navigate to its designated landmark; however, it does not know which one is its target. Instead, the target landmark is known only by the other agent. Consequently, the agents can only reach their destinations through communication, where they

exchange information about the direction they need to move (up, down, left, right, or stay still). In this way, each agent alternates between the roles of communicator and listener. The reward is computed as a combination of two components: one proportional to each agent's distance to its assigned landmark, and another based on the average distance of all agents to their respective landmarks. Although all agents share the common objective of reaching their designated landmarks—a task that requires cooperation through communication—their individual reward signals may differ. Therefore, we classify this environment as a **cooperative** problem.

Simple Speaker Listener is a variation of Simple Reference where one agent acts as the speaker, meaning it can communicate but cannot move, while the other acts as the listener, meaning it cannot communicate but must follow the speaker's instructions to reach the target. The speaker informs the listener about the direction to move (up, down, left, right, or no movement) to reach the target, and both agents receive a reward signal proportional to the listener's distance to the landmark (the closer the better). In this setup, both agents receive the same reward at each timestep, i.e the reward can be modeled using a single shared reward function. We therefore classify this environment as a **fully cooperative** problem. Additionally, this environment is characterized by the fact that agents operate over disjoint action spaces. The speaker agent has 3 available actions (`Discrete(3)`), while the listener agent has 5 available actions (`Discrete(5)`). To handle this particularity, we apply the unified action space construction described in Subsection **??**.

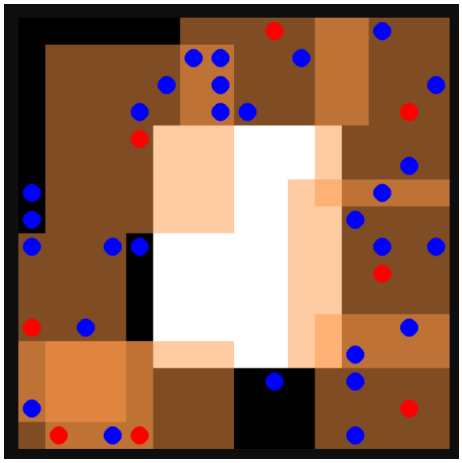
Simple Spread is an environment with N agents and N landmarks, where each agent must reach a landmark while avoiding collisions. In this environment, agents are rewarded with identical signals, determined by the minimum distance of any agent to its designated landmark. As such, we classify this setting as a **fully cooperative** problem.

4.2.3. SISL Environments

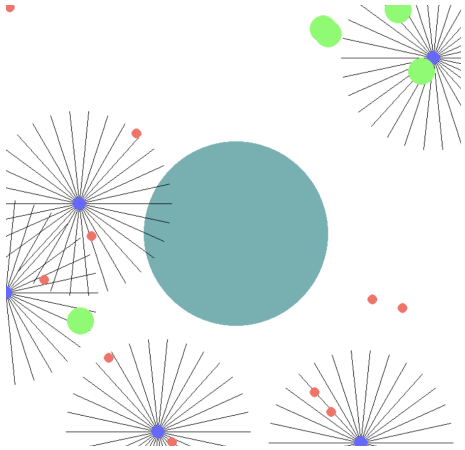
Lastly, we are going to examine the Stanford Intelligent Systems Laboratory environments, readily available at PettingZoo. These environments are a set of three multi-agent benchmark environments, **Multiwalker**, **Pursuit** and **Waterworld**. A visual example of each of these environments is shown in Figure **??**.

Multiwalker is an environment containing bipedal robots that carry a package on top of all of them. Their goal is to move to the right without the package falling to the ground. The robots receive a positive reward equal to the change in position of the package with respect to the previous instant. On the other hand, they receive an overall negative reward if the package falls to the ground and an individual negative reward if the robot itself falls to the ground. Agents also receive a small negative reward if the angle of their head changes, in order to keep their head oriented horizontally. This environment ends if the agents manage to move the package to the end of the terrain or, alternatively, if the package falls to the ground or falls off the left edge of the screen.

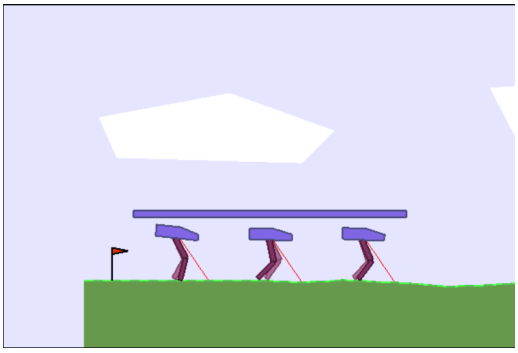
This environment features a `shared_reward` parameter that controls whether agents receive identical reward signals or individual ones. Specifically, if set to `False`, each agent receives a reward based on how far it has progressed forward (in addition to previously described penalty signals). If set to `True`, the individual reward signals are averaged, and all agents receive the same reward signal. Thus, agents work towards a common goal; however, the reward functions may be either individual or shared depending on the environment configuration. Consequently, depending on this parameter, the environment can be classified as either **fully cooperative** or **cooperative**.



(a) Pursuit



(b) Waterworld



(c) Multiwalker

Figure 4.3: Example of each of the SISL environments

Pursuit is an environment consisting of a 16x16 grid with a white obstacle in the center, where a certain number of evaders and pursuer agents are placed. The evaders move in random trajectories and the pursuer agents are the ones who must learn a policy. In Figure **??** we can see a snapshot of the environment where pursuer agents take the form of red dots, and evaders take the form of blue dots. These agents can surround an evader and, in that case, they receive a positive reward and the corresponding evader is removed from the environment. A smaller positive reward is also perceived in case a pursuer comes into contact with an evader. Each agent can move discretely down, up, left and right. The environment ends if 500 cycles are completed (default value that can be changed) or when all evaders have been eliminated. There is also an urgency reward penalty that is applied at each timestep. All in all, this environment needs individual reward functions to be modeled. However, as in the previous case, this environment has a `shared_reward` parameter that when set to `True` averages the individual reward signals of all agents and gives the same signal to all of them. We therefore conclude that this environment can be **cooperative** of **fully cooperative** depending on how it is parametrized.

Waterworld is a simulation of archaea that must move through an environment in which they can consume food but can also be poisoned. Agents are affected by the poison or get food if they are within a distance of them proportional to their own radius. This environment allows for the configuration of an `n_coop` parameter that determines how many agents must touch the food at the same time to consume it. When a number of `n_coop` agents touches the food, they all receive a certain positive reward (and the food is not immediately destroyed). On the other hand, each individual agent also receives a smaller positive reward for coming into contact with food, and a penalty for touching poi-

son. In addition, they receive a penalty depending on the Euclidean norm of their action vector. The action vector of an agent can be understood as a 2-tuple $a = (a_x, a_y)$ where each component represents the acceleration imposed on each of the two Cartesian axes. Given how agents share the goal of getting as much food as possible, but they receive individual reward signals, we conclude that this environment represents a **cooperative** problem.

4.2.4. Vectorization of image-based environments

As discussed in Subsection **??**, our supervisor agent operates using one-dimensional vectors as observations from the agents. We also noted that the Cooperative Pong and Pistonball environments are based on RGB image observations. To address this mismatch, we follow the approach proposed in [**?**], which employs a pre-trained *ResNet-18* [**?**] model trained on the *ImageNet* [**?**] dataset to extract compact vector embeddings from image-based observations. Specifically, we remove the classification head of the pre-trained network and, upon receiving an image observation, perform a forward pass through the modified ResNet-18. We then extract the output of the last fully connected layer, resulting in a fixed-size vector of 512 features. This vector is subsequently used as the observation input for our agent. The pre-trained network is loaded using the `torchvision` library [**?**], which offers convenient access to ResNet architectures trained on ImageNet.

However, given the high dimensionality of this representation—especially when compared to other environments, where observation vectors rarely exceed 100 dimensions—we explored the possibility of obtaining more compact vector representations of the agents' observations. To achieve this, we worked directly with the RGB images using OpenCV [**?**], a powerful library that provides a wide range of tools for image processing. For the Cooperative Pong environment, we developed the `CooperativePongWrapper` class. This class processes the RGB image observed by an agent to extract the (x, y) coordinates of both paddles and the ball. Additionally, it computes the ball's velocity, which provides agents with valuable information about the region of the screen where the ball is likely to move next. This approach reduces the observation vector from 512 dimensions to a much more compact vector of just 8 features.

Similarly, for the Pistonball environment, we implemented the `PistonBallWrapper` class, which also processes the agents' RGB image observations. In this case, it extracts the (x, y) coordinates of the tops of the pistons present in each observation (recall that each agent's observation includes itself and its two adjacent pistons). If the ball is visible in the observation, its (x, y) coordinates are also returned; otherwise, the coordinates $(-1, -1)$ are returned. Once again, this process reduces the dimensionality of the observation from 512 to just 8 features. In Chapter **??**, through our experiments, we aim to assess whether these handcrafted vector representations offer any advantage during the learning process compared to using embedding vectors obtained via a pre-trained ResNet-18.

4.2.5. A fully cooperative and cooperative environment classification

As a summary, Table **??** presents a classification of the PettingZoo environments into fully cooperative and cooperative problems. This categorization allows us to structure our experiments into two main groups. In the case of cooperative environments, we will apply the $SoftminMean$ function introduced in Chapter **??** to assess whether it has a positive impact on the agents' training process.

Category	Environment	Fully Cooperative	Cooperative
Butterfly	Knights Archers Zombies		✓
	Cooperative Pong	✓	
	Pistonball	✓	
MPE	Simple Reference		✓
	Simple Speaker Listener	✓	
	Simple Spread	✓	
SISL	Multiwalker	✓	✓
	Pursuit	✓	✓
	Waterworld		✓

Table 4.1: PettingZoo environments classified by cooperation type

CHAPTER 5
Experiments & Analysis

In this chapter, we present a thorough review of our experimental process. As discussed in Chapter **??**, our environments can be categorized as either cooperative or fully cooperative. The main experimental difference between these two types lies in the choice of aggregation functions appropriate for each scenario. In Chapter **??**, we stated that summation-based aggregation is sufficiently robust for fully cooperative environments—where all agents receive the same reward—but may become problematic in cooperative environments, where agents can receive different reward signals.

Building on this insight, we begin by analyzing the fully cooperative environments, where we adopt summation as our aggregation method. We then proceed to examine the cooperative environments, using summation as a baseline for comparison. In addition, we evaluate the *Softmin + Mean* approach to determine whether this alternative aggregation method leads to improved performance. In all cases, we evaluate the performance of the supervisor agent by monitoring the evolution of the *normalized average reward* over the training epochs. Specifically, we compute the total reward obtained in each episode (i.e., the sum of all aggregated rewards during an episode), average it over the last 100 episodes, and normalize the result to the $[0, 1]$ range. This metric enables us to determine whether the supervisor's performance improves throughout training.

It is important to note that this reward is normalized based on the range of reward values observed during our training loops. Therefore, the fact that the training curves reach a reward of 1.0 does not necessarily imply that the agent has achieved optimal performance. Instead, it indicates that the agent has reached the highest reward observed during training. For this reason, in Reinforcement Learning it is common to see training curves where the reward increases over time and appears to plateau around the value of 1.0, resulting in a "healthy" learning curve that suggests an improving policy. However, this peak may correspond to a local maximum and still be far from the global optimum required for truly optimal agent behaviour.

Consequently, for each experiment, we provide a link to a video where the reader can visually assess the actual performance of our supervisor meta-agent and determine whether it successfully coordinates the agents to achieve their objective. All referenced videos, as well as the code developed in the course of this work, are available in our public GitHub repository [**?**].

Finally, before proceeding, it is worth mentioning that all the experiments presented in this chapter were conducted on the *Google Colab* [**?**] platform. This platform enables online development using *Jupyter Notebooks* on machines equipped with GPUs. Specifically, we utilized instances with NVIDIA Tesla T4 GPUs and Intel(R) Xeon(R) CPUs, featuring 16 GB of VRAM and 12 GB of system RAM. Although we do not report execution times for each individual experiment, it is important to note that all of them required more than

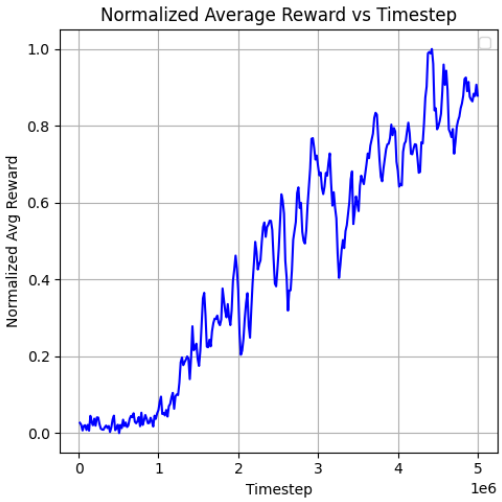
12 hours to complete, with the Pistonball environment experiments standing out, taking nearly 24 hours.

5.1 Fully Cooperative Environments

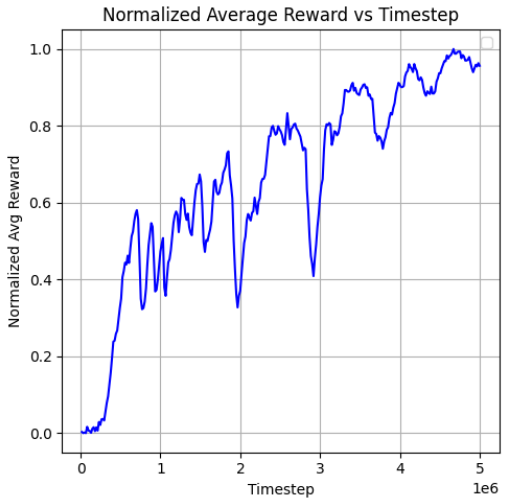
We begin by analyzing the fully cooperative environments, in which we use summation exclusively as the aggregation method. We then present the results obtained for each of these environments. For the Cooperative Pong and Pistonball environments, we additionally compare performance using RGB image observations against the vectorized representations proposed in Chapter **??**.

5.1.1. Cooperative Pong

Cooperative Pong is a variant of the classic Pong game in which agents must cooperate to keep the ball in play for as long as possible. The challenge in this environment lies in achieving coordinated behaviour that prevents the ball from "escaping" the playing field. By default, observations in this environment take the form of RGB images—snapshots of the playing field that include both paddles and the ball. The training results using RGB image observations (which, it should be recalled, are processed by a ResNet-18 network to extract an embedding vector) can be seen in Figure **??**. Alternatively, we proposed a vectorized representation that converts each observation into an 8-dimensional feature vector. The training results using this vectorized input can be found in Figure **??**.



(a) Cooperative Pong: RGB based observations



(b) Cooperative Pong: vector based observations

Figure 5.1: Training curves for Cooperative Pong - Sum Aggregation

In this case, we observe that both RGB image observations and vectorized observations lead to a clear improvement in the performance of the supervisor agent. Specifically, training with vectorized observations appears to produce a more abrupt increase in performance during the earlier stages of training compared to the image-based approach. This faster rise may be attributed to the lower complexity of the vectorized inputs, which allows the agent to learn a better policy sooner.

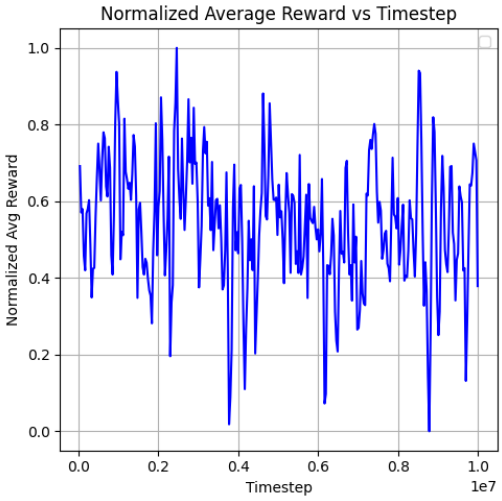
In both cases, we notice dips in the performance curves, indicating temporary decreases in reward. Such declines are characteristic of optimization algorithms like PPO, which may, during certain phases of training, exploit a suboptimal policy resulting in

reduced performance, or enter exploration phases where the algorithm searches for alternative policies that momentarily lower the reward.

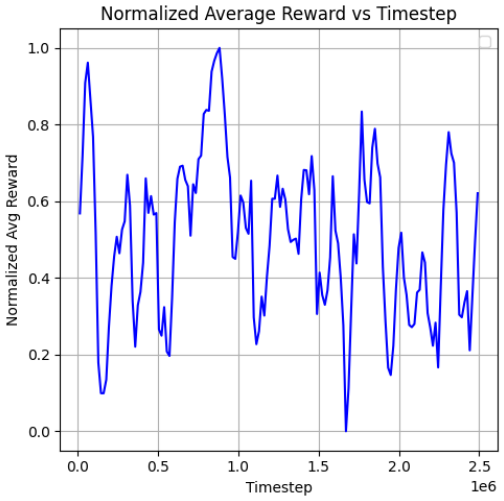
Overall, both approaches show significant gains in the supervisor agent's performance. We therefore conclude that even high-dimensional observations, such as those extracted via a ResNet-18, provide sufficiently discriminative and informative inputs to enable successful training of the supervisor. Ultimately, both methods lead to the development of a meta-agent supervisor that enables both paddles to cooperate effectively and achieve their shared objective. The user can review videos available at the Cooperative Pong section to check the supervisor agent performance. As can bee seen, both approaches manage to keep the ball on the playing field until the game ends.

5.1.2. Pistonball

Pistonball is an environment where certain pistons (agents) can move up or down and must cooperate to move a ball as quickly as possible to the left side of the stage. Again, the default observations take the form of RGB images, and we also propose a vectorized version consisting of 8-dimensional feature vectors. The training results using RGB images can be seen in Figure **??**, while the results for the vectorized version are shown in Figure **??**.



(a) Pistonball: RGB based observations

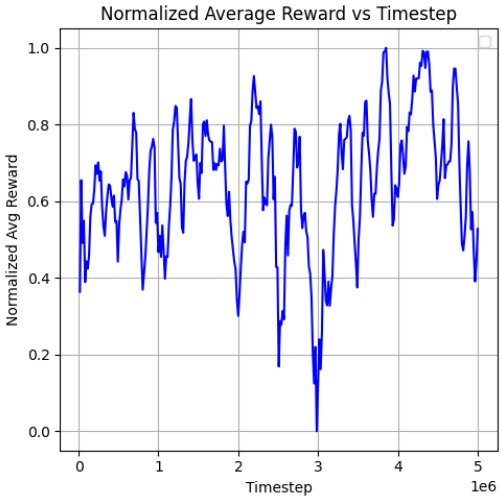


(b) Pistonball: vector based observations

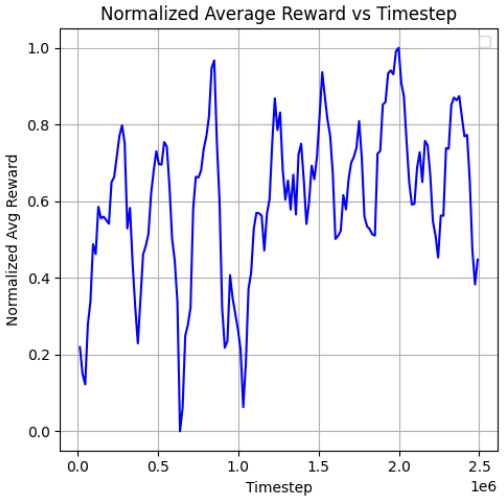
Figure 5.2: Training curves for Pistonball - Sum Aggregation

In this case, neither training approach appears to converge toward a policy that significantly outperforms the initial one. It is important to note that by initial policy we refer to the policy induced by the randomly initialized network weights at the beginning of PPO training. As a result, this initial policy is mostly stochastic. The objective of training the supervisor meta-agent is to learn a policy that approximates an optimal one and, ultimately, diverges significantly from this initial random behaviour. As shown in the training curves, the normalized average reward fluctuates widely—ranging from values close to 0.0 up to 1.0—and, in some instances, the resulting policies even underperform compared to the initial policy. The most notable difference between the two curves lies in the increased noisiness and higher frequency of oscillations observed when using image-based inputs. This instability may be attributed to the greater complexity of the visual features extracted by ResNet-18.

Overall, this environment does not seem to train satisfactorily regardless of the observation method used. We therefore conclude that the difficulty in this environment stems



(a) Pistonball: RGB based observations



(b) Pistonball: vector based observations

Figure 5.3: Training curves for Pistonball with Discrete action space - Sum Aggregation

from the inherent complexity of the task and the challenge of coordinating a larger number of agents. In our experiments, we use 8 agents (pistons), a significantly higher number compared to environments like Cooperative Pong, which involves only two agents. Additionally, the action space in this case is continuous rather than discrete. Typically, the use of continuous actions adds complexity to the policy that the Actor must learn in our PPO algorithm. In summary, Pistonball does not appear to explicitly benefit from the centralized approach we propose. As before, the performance of the supervisor agent can be assessed through the videos available at the Pistonball section. These recordings clearly demonstrate how, in this case, the agent fails to achieve the desired behaviour.

However, after reviewing the available documentation for this environment, we found that it accepts a boolean parameter, `continuous`, which determines whether the action space is continuous or discrete. By setting this parameter to `False`, the action space becomes `Discrete(3)` i.e., three discrete actions corresponding to moving up, staying still, or moving down. Accordingly, we repeated the same experiments using a discrete action space instead of a continuous one. The results of this variation are shown in Figure **??**. Despite this modification, the results remain comparable to those obtained in the continuous case. The training process still exhibits pronounced oscillations that persist throughout the entire training loop. Once again, the reader can find videos demonstrating the behavior of the supervising meta-agent for both RGB image observations and vectorized observations at the Pistonball Discrete section.

In conclusion, based on these results, we cannot assert that the continuous nature of the action space is the primary challenge in this environment. Since both configurations yield remarkably similar outcomes, it appears that the task itself is inherently complex, and our centralized approach struggles to achieve satisfactory performance under such conditions.

5.1.3. Simple Spread

Simple Spread is an environment in which three agents must each cover one of three landmarks placed across the scenario. In this case, observations are provided directly as one-dimensional vectors, and therefore no additional preprocessing is required. The results of this experiment are shown in Figure **??**.

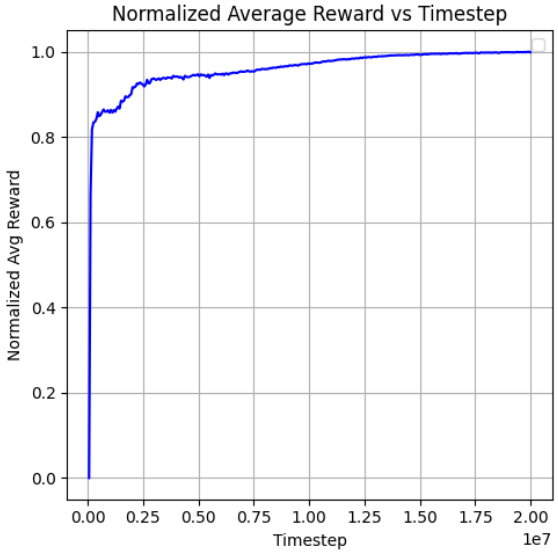


Figure 5.4: Training curves for Simple Spread - Sum Aggregation

It is clear that the supervisor meta-agent achieves excellent performance in this task, reaching near-optimal behaviour quickly and with minimal oscillation. A video of this seemingly optimal behaviour can be found in our repository under the Simple Spread section, where it is evident that the agents effectively learn to coordinate and cover all landmarks.

We conclude that our centralized approach enables efficient coordination among agents, allowing them to successfully achieve their shared goal of covering all landmarks in the environment.

5.1.4. Simple Speaker Listener

Simple Speaker Listener is an environment where one agent acts as a listener while another agent—the speaker—must guide it toward a target landmark. The learning curve, displaying the normalized average reward, is shown in Figure **??**. In this case, we observe a training process that, at first glance, resembles the one in the previous scenario: the supervisor learns rapidly during the initial stages of training, after which performance appears to plateau—until a final stage where a sudden spike in performance is observed.

If the reader examines the video located at the Simple Speaker Listener section, they will see that despite this seemingly healthy training curve, the agent's behaviour exhibits mixed results. In some instances, the listener manages to approach its target landmark successfully (that is, the agent represented as a coloured ball approaches another ball of the same colour). However, in other episodes, the listener initially heads in the right direction but eventually strays and fails to reach the intended goal.

As discussed in Chapter **??**, this environment involves disjoint action spaces, which we addressed by constructing a unified action space, `Discrete(15)`, derived from the Cartesian product of both agents' individual action spaces. While 15 actions may seem few, most environments we deal with typically involve smaller action spaces, ranging from 5 to 7 actions (or even just 3 in the case of Cooperative Pong). In the upcoming experiments, we will explore whether dealing with larger action spaces poses a challenge for our centralized approach. Based on these observations, we conclude that the Speaker Listener environment does not appear to benefit significantly from our centralized method.

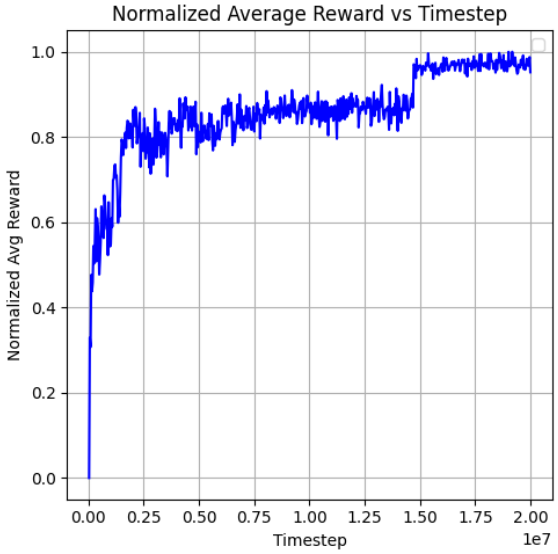


Figure 5.5: Training curves for Simple Speaker Listener - Sum Aggregation

5.1.5. Pursuit

Pursuit is an environment in which a group of pursuer agents must cooperate to capture a set of evaders. To do so, they need to coordinate in such a way that they successfully surround the evaders, as this is the only way to eliminate them and make them disappear from the environment. The goal, therefore, is to eliminate as many evaders as possible. In Figure **??**, we show the evolution of the normalized average reward during training. In this case, the training curve suggests a clear learning behaviour, with performance improving significantly over time and no major oscillations. However, as the reader can observe in the video available at the Pursuit section, while the agents manage to get close to the evaders, the number of successful captures (i.e., coordinated eliminations) remains low (this can be inferred from the fact that the number of blue dots remains relatively constant over the course of the game).

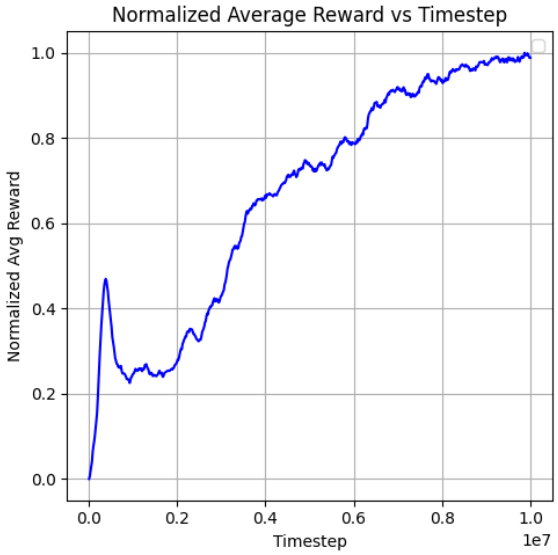


Figure 5.6: Training curves for Pursuit - Sum Aggregation

This behaviour may originate from the reward function itself, as agents receive a small positive reward whenever they approach an evader. Consequently, the supervi-

sor appears to have learnt to keep the pursuers in close proximity to the evaders, without effectively coordinating them to encircle and eliminate the targets. This is a well-known phenomenon in Reinforcement Learning, where the learnt policies can yield significant increases in cumulative reward without necessarily achieving the intended or optimal behaviour. In the following section, we investigate whether adopting a more explicitly cooperative approach to the task leads to improved performance.

5.1.6. Multiwalker

Multiwalker is an environment in which two robots must cooperate to move forward without dropping a plank that they carry above their heads. Figure **??** shows the progression of the normalized average reward throughout the training process. In this case, we observe a situation similar to that of Pistonball: the training is highly noisy, with significant oscillations in the average reward. Overall, it appears that the training does not lead to a policy that meaningfully improves upon the initial one.

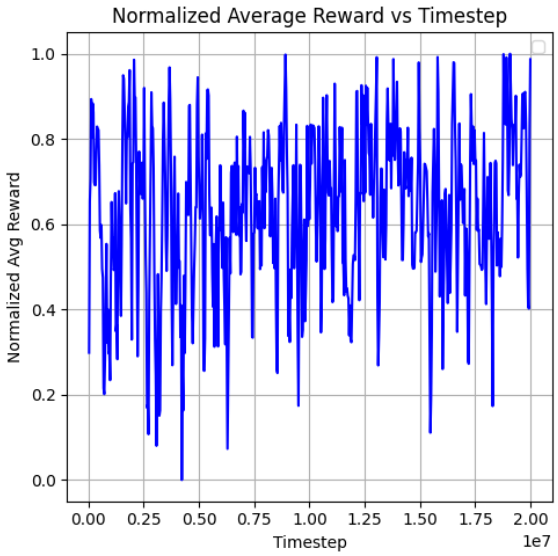


Figure 5.7: Training curves for Multiwalker - Sum Aggregation

By examining the video located at the Multiwalker section, we can see that the agents manage to keep the plank balanced above their heads, but at the cost of barely moving forward or even moving their heads at all. Clearly, this reflects a highly suboptimal policy—one that is far from achieving the intended behaviour of coordinated forward movement without dropping the plank.

As in the Pistonball environment, this setting relies on continuous actions of the form `Box(-1, 1, (4,))`—that is, 4-dimensional vectors controlling the joint forces of each agent's two legs. As we will discuss in Section 5.3, this characteristic may introduce certain challenges for our centralized approach. In the next section, we examine whether Multiwalker can benefit from a cooperative strategy by assigning distinct rewards to each agent, or whether performance remains limited.

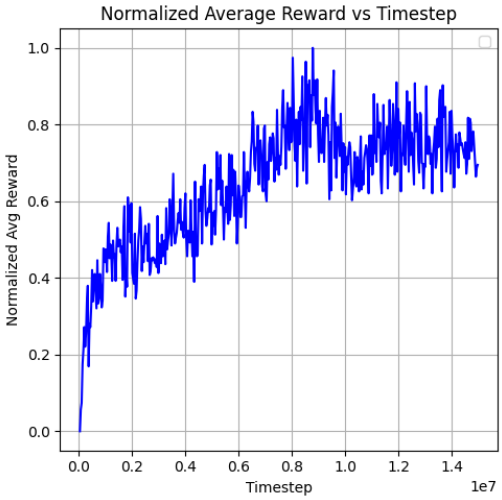
5.2 Cooperative Environments

In this section, we analyze the results obtained in cooperative environments. Given that agents can now receive distinct reward signals, we employ the Softmin + Mean aggregation method to combine individual reward vectors, with the goal of evaluating the

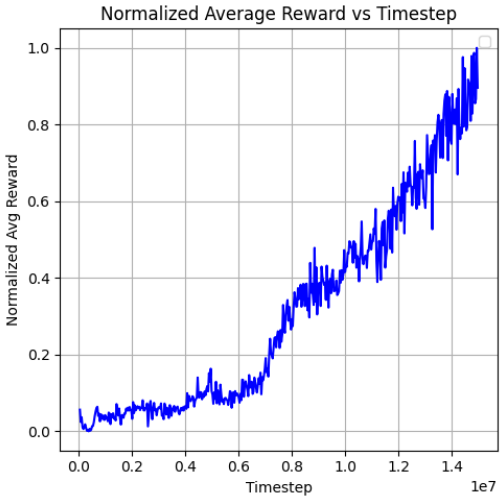
performance of the centralized supervisor. For comparison, we also include the Sum aggregation method as a baseline, to assess whether the proposed alternative approach leads to a meaningful improvement in the performance of the supervisor meta-agent.

5.2.1. Knights, Archers & Zombies

Knights, Archers & Zombies is an environment where two types of agents—knights and archers—must cooperate to eliminate waves of zombies that appear throughout the scenario. As mentioned at the beginning of this section, Figure **??** presents the training curves obtained using two different reward aggregation methods: Sum and Softmin + Mean. In this case, we observe that training with the Sum aggregation method appears to lead to improved performance as training progresses. However, this improvement is accompanied by considerable noise, with strong fluctuations that persist throughout the entire training process.



(a) Knights, Archers & Zombies: Sum aggregation



(b) Knights, Archers & Zombies: Softmin + Mean aggregation

Figure 5.8: Training curves for Knights, Archers & Zombies

On the other hand, training using the Softmin aggregation method shows a significant improvement in performance, with far less pronounced fluctuations compared to the Sum aggregation case. However, upon reviewing the videos available at the Knights, Archers & Zombies section, we can observe substantial differences in agent behaviour and overall performance. In the Sum aggregation case, the agents display relatively poor cooperation. Although both of them attempt to contribute to eliminating zombies—especially the archer, who dispatches the majority—this cooperation proves insufficient. Frequently, one or more zombies manage to escape early in the episode, reaching the bottom of the screen and ending the game prematurely.

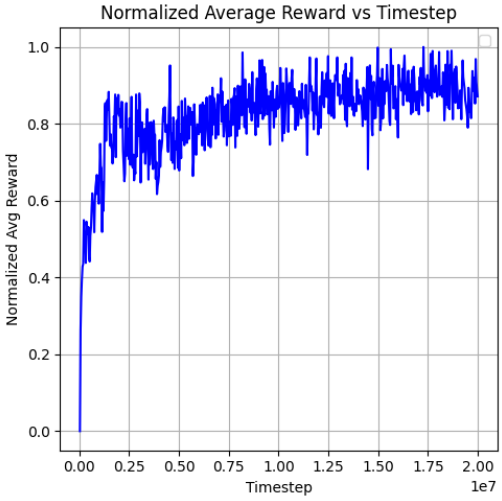
In contrast, training with Softmin aggregation results in two noteworthy behaviours. First, the episodes tend to last much longer, indicating that the agents are able to eliminate a substantial number of zombies before one eventually breaks through. Second, during gameplay, the archer appears to handle nearly all enemies, while the knight is typically eliminated early in the episode. This suggests that the learnt policy heavily favours the archer, who ends up performing the majority of the task. We attribute this to the intrinsic advantages of the archer: it can eliminate zombies from a distance, whereas the knight must get dangerously close. Once a knight comes into contact with a zombie, it is immediately eliminated. Therefore, the knight's learning process is inherently more

complex, as the supervisor must discover a policy that positions the knight close enough to strike, yet far enough to avoid being taken out.

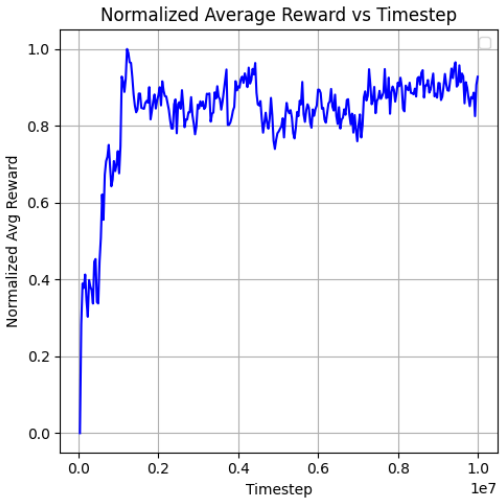
In conclusion, we find that this particular environment significantly benefits from our centralized supervision approach, provided that reward vectors are aggregated carefully—specifically using the Softmin + Mean method.

5.2.2. Simple Reference

Simple Reference is an environment in which two agents must reach their respective target landmarks through mutual communication. Specifically, each agent is unaware of the location of its own landmark but knows the location of the other agent's target. Thus, both agents must act simultaneously as speakers and listeners in order to reach their destinations. The training curves for both reward aggregation methods are shown in Figure **??**. In both cases, we observe similar behaviour: an early rise to peak performance, followed by a "maintenance" phase where performance plateaus without substantial improvement.



(a) Simple Reference: Sum aggregation



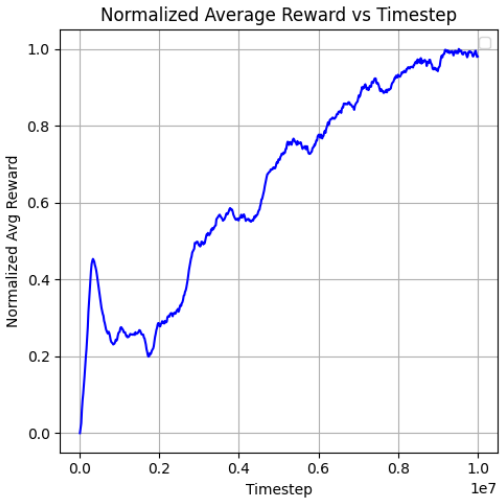
(b) Simple Reference: Softmin + Mean aggregation

Figure 5.9: Training curves for Simple Reference

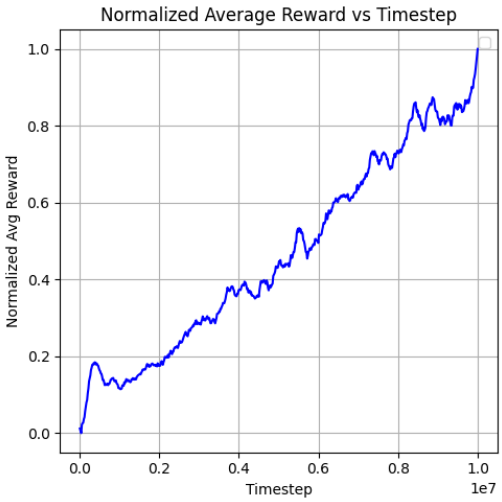
However, after reviewing the videos Simple Reference section, we observe erratic agent behaviour in both cases. In neither case do the agents consistently reach their target landmarks; more often, they wander around the environment without moving closer to their goals. According to the documentation provided by PettingZoo for this environment, the action space takes the form `Discrete(50)`. This is because each action encodes one of five possible movement directions (up, down, left, right, or no movement) combined with one of ten possible communication signals. As such, the overall action space is the Cartesian product of these two sets. In summary, as in Simple Speaker Listener, we are dealing with a large action space, which may pose challenges for our centralized approach. In the following section, we will examine how larger action spaces can impact the supervisor's performance. All in all, we conclude that Simple Reference does not seem to benefit from our centralized method.

5.2.3. Pursuit

We now propose training the supervisor meta-agent by configuring Pursuit as a cooperative environment. As discussed in the previous chapter, this is achieved by setting the `shared_reward` parameter to `False`, so that agents receive individualized rewards based on their actual contribution to the shared objective of capturing evaders. In Figure **??**, we show the evolution of the normalized average reward during training for both Sum and Softmin + Mean aggregation. Once again, as in the previous section, the training curve demonstrates a clear improvement in the performance of the supervisor meta-agent for both cases, with a significant increase in reward over the course of the training epochs.



(a) Pursuit: Sum aggregation



(b) Pursuit: Softmin + Mean aggregation

Figure 5.10: Training curves for Cooperative Pusuit

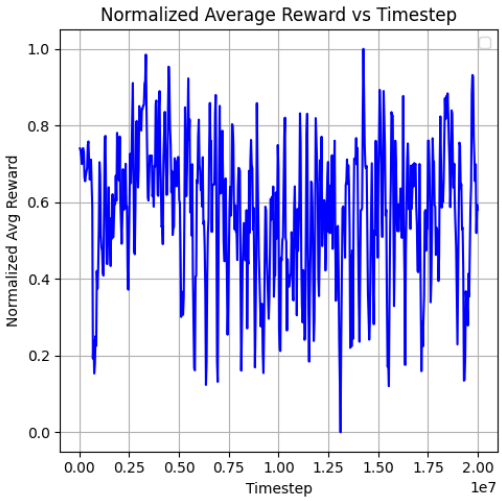
However, if the reader examines the videos available at Pursuit section, they will notice that the agents' in-game performance is substantially better compared to the previous case. On the one hand, the video that corresponds to the training with Sum aggregation shows the same flaws as in the previous section. That is, agents do not seem to learn how to successfully surround evaders and effectively eliminating them. On the other hand for the Softmin + Mean case, watching the video from beginning to end (patiently, as episodes can be lengthy), it becomes evident that the number of evaders (blue dots) decreases significantly as the episode progresses. This behaviour was not observed when using sum-based aggregation, despite the seemingly "healthy" training curves. In this case, the use of this aggregation method appears to have enabled the supervisor agent to learn a policy that effectively coordinates the agents, allowing them to surround the evaders and eliminate them decisively.

This leads us to conclude that Pursuit is an environment that can benefit from our centralized approach, provided that reward vectors are aggregated carefully using the Softmin + Mean function.

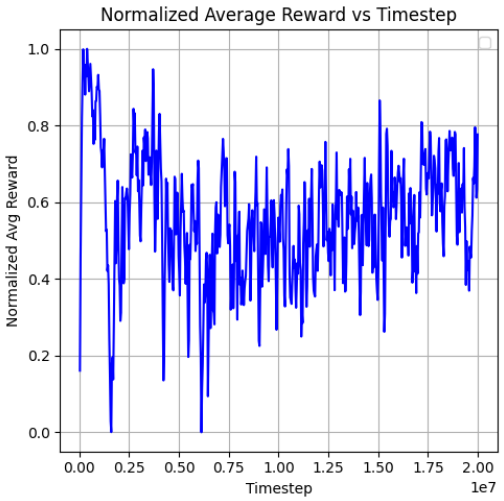
5.2.4. Multiwalker

We now propose training the supervisor meta-agent by configuring Multiwalker as a cooperative environment. As discussed in the previous chapter, this is achieved by setting the `shared_reward` parameter to `False`, so that agents receive individualized rewards based on their actual contribution to the shared objective of capturing evaders. In Figure **??**, we show the evolution of the normalized average reward during training for both

Sum and Softmin + Mean aggregation. Once again, we observe behaviour very similar to that seen in the previous section with this same environment. As shown in the Figure, both curves exhibit a noticeably noisy training process, with reward oscillations persisting throughout all training epochs. In the end, the supervisor meta-agent does not appear to learn a policy that significantly diverges from the initial one.



(a) Multiwalker: Sum aggregation



(b) Multiwalker: Softmin + Mean aggregation

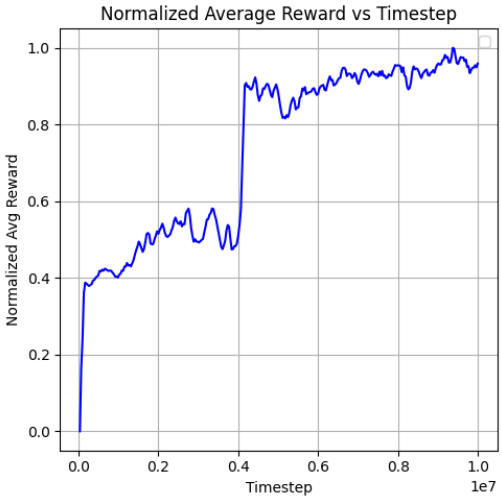
Figure 5.11: Training curves for Cooperative Multiwalker

By reviewing the videos available at the Multiwalker section, the reader can verify that the robots fail to complete their intended task. In the case of training with the Softmin + Mean reward scheme, the agents appear to take a more active role compared to the sum-based reward, rather than remaining completely static, they attempt to move forward in a coordinated fashion. However, as shown in the video, these attempts consistently result in the plank falling to the ground, thereby terminating the episode. In summary, Multiwalker is an environment that does not seem to benefit from our centralized approach.

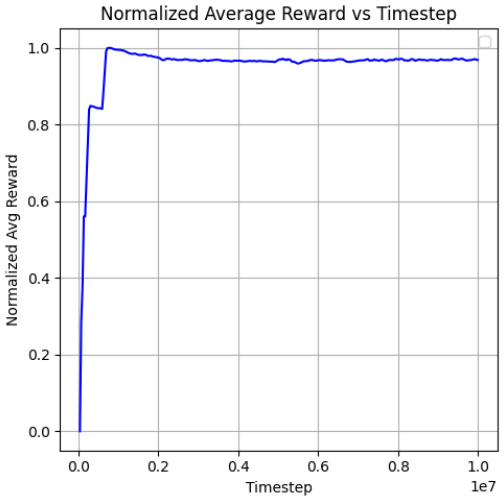
5.2.5. Waterworld

Waterworld is an environment where a group of archaeas must navigate a space, consuming food while avoiding poison. Food, represented by green circles, can only be successfully consumed when two archaeas are in close proximity to it simultaneously. In Figure **??**, we present the training curves for the two reward aggregation strategies. In the case of the Sum strategy, training exhibits two significant "jumps" in performance, suggesting that the supervisor may have discovered policies that substantially improve agent behaviour. For Softmin + Mean, there is a sharp increase in performance during the early stages of training, followed by a plateau, with little improvement beyond the first million timesteps. Overall, both strategies appear to produce a significant improvement in policy quality compared to the initial stages of training.

However, upon reviewing the videos available at the Waterworld section, distinct behavioural patterns can be observed. In the Sum case, although the agents have learnt to approach the food (green circles), as they receive small positive rewards for contact, they have not learnt the need to cooperate with one another to consume it successfully. In contrast, under the Softmin + Mean scheme—though still far from optimal—once the agents make contact, they tend to remain together or in close proximity for extended periods, thereby enabling more consistent and successful food consumption.



(a) Waterworld: Sum aggregation



(b) Waterworld: Softmin + Mean aggregation

Figure 5.12: Training curves for Waterworld

We therefore conclude that Waterworld is an environment that can benefit from our centralized approach, provided that the reward aggregation is carefully handled using methods such as Softmin + Mean. Recall also that, as discussed in Chapter **??**, this environment uses continuous actions in the form of two-dimensional vectors, representing acceleration along both axes. In this case, despite dealing with continuous action spaces, we observe that the environment achieves notable performance—unlike other environments such as Pistonball or Multiwalker.

5.3 General Discussion

In this section, we present a general discussion of the results obtained during our experimental phase.

5.3.1. Success cases for the Softmin + Mean approach

To initiate the discussion, it is worth noting that **in cooperative environments** such as Knights, Archers & Zombies, Pursuit, and Waterworld, **our reward vector aggregation strategy, Softmin + Mean, consistently demonstrates superior performance compared to simple summation**. We believe this is because summation, under certain circumstances, can lead to an artificial inflation of the scalar reward signal. For example, in environments like Pursuit and Waterworld, agents receive a small reward signal when approaching their goal (e.g., getting closer to an evader in Pursuit or to food in Waterworld) and a larger signal when they successfully cooperate to achieve the goal (e.g., surrounding an evader in Pursuit or both archaeas reaching the food in Waterworld). However, if agents merely approach the goal and we aggregate their reward vectors by summation, the resulting reward signal may increase sufficiently for the supervisor to learn, thus raising the average reward without the agents necessarily discovering or learning the cooperative behaviour we aim to promote.

Consider, for instance, Pursuit. If each of four agents receives a reward of +1 for coming into contact with an evader and +5 for successfully surrounding it, then if all agents merely make contact, the aggregated signal would be $SumAggregation(\mathbf{r}) = 4$, even if they are scattered and far from achieving the cooperative configuration required to sur-

round the target. Even if only two agents approach an evader while the others do not contribute, we would still obtain $SumAggregation(\mathbf{r}) = 2$. This example illustrates how summation can yield deceptively high reward signals even when agents fail to cooperate or when some agents contribute nothing to the total accumulated reward.

In contrast, with Softmin + Mean, if all agents approach the goal but fail to surround it, the resulting signal would be $SoftminMean(\mathbf{r}) \approx 1$, which is less significant than in the summation case. Furthermore, if even one agent receives no reward, then $SoftminMean(\mathbf{r}) \approx 0$, imposing a stronger penalty than summation would. In short, this approach demands a higher degree of coordination among agents to produce meaningful reward signals. As a result, the meta-agent supervisor is encouraged to learn policies that yield more synchronized and cooperative behaviours which is precisely the type of behaviour required in the aforementioned environments.

On the other hand, the results in Knights, Archers & Zombies are particularly noteworthy. Recall that in this environment, agents receive a reward of +1 when they successfully eliminate a zombie, and 0 at all other times. When using sum aggregation, a plausible outcome is that one agent performs most of the work while the other contributes little or nothing. Under such conditions, the aggregated reward signal would still be +1, which is sufficient for the supervisor to reinforce and learn that behaviour. Indeed, video analysis confirms this hypothesis: the archer agent performs the majority of the task, while the knight agent fails to eliminate many zombies. Although with Softmin + Mean we would expect to observe a different behaviour-since when one agent contributes significantly and the other does not, the resulting signal would be approximately 0-we still observe a very similar outcome. In the early stages of the episodes, the knight is eliminated, and the archer completes the task alone. As discussed in the previous section, the knight's behaviour is inherently more difficult to learn than the archer's. This seems to have led the optimization process to favour eliminating the knight early on, allowing the archer to handle the entire task.

5.3.2. Are continuous actions spaces a limiting factor?

On the other hand, we experimented with environments featuring continuous action spaces such as Pistonball, Multiwalker or Waterworld and obtained mixed results. In the cases of Pistonball and Multiwalker, the outcomes were notably poor: the training curves exhibited strong oscillations that persisted throughout training, and the agents failed to discover any policy significantly better than the initial one. In contrast, Waterworld presented a completely different scenario, with much more stable training curves that quickly converged to a better-performing policy. Particularly in the case of the Softmin + Mean training variant, we observed more efficient behaviours, as evidenced in the corresponding video recordings. The reasoning behind the improved performance of this environment with our approach is very similar to what we reasoned with Pursuit in the previous subsection, as both environments exhibit highly similar dynamics. In short, the Softmin + Mean method demands a higher degree of coordination among agents in order to obtain meaningful reward signals. This, in turn, encourages the learning of policies that produce the desired cooperative behaviours.

Additionally, in Pistonball, training with discrete action spaces did not lead to significant improvements, as the agent behaviour remained very similar to that observed with continuous actions. These observations suggest that, although continuous action spaces are often associated with increased training complexity, in our experiments they do not appear to be the primary bottleneck affecting the performance of the supervising meta-agent.

Instead, we believe the poor performance in Pistonball and Multiwalker stems from the inherent complexity of the tasks. In Pistonball, agents must cooperate in such a way that one agent's action does not negate the progress made by others. For example, if the pistons collectively push the ball to the left, a single piston lifting at the wrong time can cause the ball to bounce back to the right, undoing the group's effort. In short, the pistons must learn a highly coordinated behaviour that is, raising in a specific order, to effectively slide the ball to the left without blocking its movement. Similarly, in Multiwalker, as illustrated by the linked videos, the task demands a high degree of coordination between the robots to move without letting the platform fall. For instance, if one robot moves forward while another lags behind, the platform may lose balance and collapse. In some cases, even a slight tilt of one robot's head is enough to topple the platform, regardless of the other agent's actions.

In contrast, in the Waterworld environment, the actions of a single agent do not penalize global performance as heavily. Here, agents must simply stay close to one another to maximize the reward obtained when consuming food. A seemingly "suboptimal" action at a given moment does not end the episode or undo previous progress, since the agents can always regroup and continue seeking food to optimize their joint reward signal. In conclusion, environments that demand a high degree of coordination where a single agent's action can significantly degrade overall performance, can pose a substantial challenge for our centralized meta-agent approach.

5.3.3. Limitations of Sparse and Partially Shared Rewards

Another limitation that may affect our approach is the sparsity of rewards inherent to our centralized model. Recall that, in Algorithm **??**, at each step, the meta-agent sequentially assigns actions until the full joint action is constructed and executed in the environment, where the length of the joint action directly depends on the number of agents involved. During the action assignment process, the reward received is always zero; a potentially non-zero reward is only obtained after the complete joint action is executed. Therefore, in environments like Pistonball, which involve 8 agents, there are a **total of 7 computation steps in which the reward will always be zero**. In general, **as the number of agents increases, so does the number of steps during which the reward remains zero**. This form of reward delivery may lead to sparse reward signals, particularly in environments with a large number of agents, which can in turn hinder the training of the meta-agent. We believe this to be the case, and previous works [**?**, **?**] have explicitly shown that environments with sparse or infrequent reward signals significantly degrade the performance of reinforcement learning algorithms such as PPO. In summary, this could represent a potential limitation of our model.

Regarding the Multiwalker environment, we also observed that the Softmin + Mean approach does not seem to lead to improved agent behavior. Upon closely examining the documentation provided by PettingZoo, it becomes clear that—even when the `shared_reward` parameter is set to `False`, thereby enforcing a cooperative scenario with disjoint rewards—the agents still receive the **same positive reward when successfully moving the platform forward**. Moreover, agents also receive an identical penalty if the platform falls to the ground, which causes the episode to reset. The only individual rewards they receive occur when the angle of their head deviates (encouraging them to keep it upright) or when they fall, which results in their elimination from the environment. Consequently, any potential differences in the agents' rewards will be primarily determined by variations in their head angles. Therefore, once the supervisor meta-agent learns to keep the head as stable as possible and minimize angle fluctuations, the rewards perceived by the agents will be approximately the same.

Under these conditions, even when rewards are explicitly set to be disjoint, many of them are still distributed jointly. This creates a suboptimal scenario for applying the Softmin + Mean method. As previously discussed, this aggregation strategy is most effective when rewards are truly disjoint, as it strongly penalizes agents that underperform relative to others. However, when agents receive very similar rewards at each timestep, Softmin + Mean behaves similarly to simple summation, making similar results between the two approaches likely. Additionally, forward movement of the robots typically involves a significant change in head angle. As a result, when attempting to move the robots forward, the meta-agent receives a positive reward signal due to the progress made, but also a considerable negative signal due to the variation in head angle. In other words, this reward structure, combined with the inherent complexity of the task, can lead to notable difficulties during the learning process.

In conclusion, this environment does not seem to exhibit the characteristics needed to benefit from our method. In future work, it may be worthwhile to explore *reward shaping* [**?**] techniques to assess whether Softmin + Mean can outperform the summation baseline under more suitable reward configurations.

5.3.4. On the Complexity of Continuous Action Spaces

Even if the continuous action spaces do not seem to be the main bottleneck with some environments, it is important to note that continuous action spaces can be more difficult to train and to achieve satisfactory performance. First, it should be understood that training with discrete actions can be reduced to a classification problem for the actor network. Specifically, the policy π learnt by the supervising meta-agent is modeled as a categorical distribution over a finite set of discrete actions. Typically, a *Softmax* activation is applied to the output layer of the actor network—whose size matches the number of available actions—to produce a probability distribution, from which an action is sampled.

In contrast, when working with continuous action spaces, the policy is usually modeled as a Gaussian distribution parametrized by a mean μ and a variance σ. Selecting an action therefore involves sampling from this normal distribution. Therefore, continuous action spaces may introduce additional difficulties due to the nature of the exploration process. Unlike discrete spaces, which involve selecting from a finite set of actions, continuous action spaces require the agent to explore over an infinite set of possible values. This greatly increases the complexity of exploration, as the agent must rely on stochastic sampling from continuous distributions whose variance must be carefully tuned. If the exploration noise is too small, the policy may fail to discover better actions; if it is too large, the learning process can become unstable or inefficient. These factors make policy optimization in continuous domains potentially more delicate than in discrete ones.

Additionally, reinforcement learning algorithms such as PPO can be sensitive to the choice of hyperparameters [**?**], adding an additional layer of complexity to our experiments. Future work could explore hyperparameter search strategies aimed at improving the performance achieved in these continuous-action environments. In summary, the relatively poorer results observed in these tasks may be attributed, on the one hand, to the inherent complexity of the environments themselves and, on the other, to the limited computational and time resources available, which prevented us from conducting a thorough hyperparameter optimization that might have led to better outcomes.

5.3.5. Dealing with large action spaces

Finally, regarding the Simple Reference environment, we also observed relatively poor performance, as evidenced by the videos linked in the previous section. As we noted, this environment features an action space of size 50 (`Discrete(50)`), which, compared to the other environments, makes it the largest discrete action space considered in our experiments. Prior work [**?**] has highlighted the challenges associated with reinforcement learning in large discrete action spaces, and it is plausible that our supervising meta-agent struggles to cope effectively with this increased complexity.

It is also worth noting that, in this case, our Softmin + Mean approach does not yield a substantial performance improvement over simple summation. Recall that in this environment, each agent receives a reward proportional to its distance from its assigned landmark—the closer it gets, the higher the reward. This reward structure initially suggested that Softmin-based aggregation would be particularly well-suited to the task. In principle, if one agent performs significantly worse (i.e., remains far from its landmark), the Softmin would heavily penalize this behaviour, whereas the Mean would still assign partial credit for the improvement of other agents. Despite the apparent suitability of this method for handling such disjoint reward structures, the behaviours observed in the previously linked videos remain far from optimal for this task.

Once again, we argue that the large discrete action space the supervisor must handle represents a potential limitation of the model, hindering its ability to achieve satisfactory performance regardless of the aggregation method employed. Future work could explore the meta-agent approach in environments with progressively expanding action spaces, aiming to assess whether this methodology consistently struggles under such conditions and to identify strategies for alleviating these limitations.

5.3.6. Main lessons learned

To summarize the key points discussed throughout this chapter, we present the following general conclusions drawn from the work carried out:

- **Successful applications of the centralized approach**: Based on our results, we conclude that our centralized meta-agent supervisor approach **performs well in both cooperative and fully cooperative tasks**. Environments such as Cooperative Pong, Knights, Archers & Zombies, Pursuit, Waterworld, and Simple Spread appear to benefit from our method, as can be observed in the videos linked in the previous section. Consequently, we believe our approach shows promising results for multi-agent reinforcement learning problems, potentially offering improvements in terms of scalability and efficiency compared to distributed alternatives.

- **Softmin + Mean as a reward aggregation function**: Our experiments reveal that **cooperative environments** such as Pursuit, Waterworld, and Knights, Archers & Zombies **clearly benefit from the use of the Softmin + Mean aggregation strategy**, often outperforming the standard summation approach. This reinforces the idea that, in multi-agent settings where agents share a common objective but receive separate reward signals, the choice of reward aggregation function plays a crucial role in shaping the learning dynamics of the supervisor. By attenuating the influence of extreme or uninformative rewards while still capturing the collective performance, Softmin + Mean provides a more nuanced scalar signal that facilitates more stable and effective learning. These results not only highlight the practical relevance of principled aggregation strategies but also **open the door to the**

exploration of new reward aggregation methods that could further enhance the performance and robustness of the supervisor meta-agent.

- **Environments with high coordination requirements**: In environments such as Pistonball or Multiwalker, which require a high degree of inter-agent coordination and where suboptimal actions from a single agent can significantly hinder collective performance, our centralized approach appears to face notable challenges. These findings suggest that the effectiveness of our method may be sensitive to the level of agent coupling and interdependence. Future work should further investigate this hypothesis by evaluating the model in similarly demanding environments, in order to assess whether such coordination-intensive scenarios expose a systematic limitation of the proposed approach.

- **Continuous action spaces**: While our approach has demonstrated the ability to learn and perform competitively in environments with continuous action spaces—such as Waterworld—these settings remain particularly challenging. Unlike discrete action spaces, continuous ones require exploration and optimization over an uncountably infinite range, increasing the complexity of both policy learning and convergence. Further research could examine whether incorporating specialized techniques for continuous control—such as action space discretization or adapted exploration strategies—might enhance performance in these environments.

- **Reward sparsity during joint action construction**: As observed, during the process of constructing joint actions, the reward signal remains zero at every intermediate step. A potentially non-zero reward is only received once the full joint action has been constructed and executed in the environment. This leads to a sparse and delayed reward signal, which can hinder the learning dynamics of the supervisor meta-agent. Such sparsity is considered an intrinsic limitation of the model, and may pose significant challenges—particularly in environments with a large number of agents, where the number of intermediate steps with zero reward increases accordingly.

- **Hyperparameter sensitivity**: PPO is known to be highly sensitive to the choice of hyperparameters. Due to time and computational constraints, this work did not include an extensive hyperparameter search. Future work could focus on a more thorough hyperparameter optimization process to determine whether this algorithm is capable of fi°nding solutions closer to the global optimum, and ultimately improving the behaviour of the supervisor meta-agent.

- **Large action spaces**: Our approach seems to encounter difficulties in environments such as Simple Reference, which features a significantly larger action space (`Discrete(50)`) compared to the other environments considered. This suggests that the performance of our centralized method may degrade as the action space expands, possibly due to the increased complexity of the action selection process for the meta-agent. We identify this as a potential limitation of the proposed approach. Future research could systematically investigate this aspect by evaluating performance across environments with progressively larger action spaces, in order to determine whether this behavior reflects a consistent and inherent constraint of the method.

- **Image-based observation environments**: In the Cooperative Pong environment, we observed that employing a ResNet-18 model pretrained on ImageNet enabled the extraction of meaningful visual embeddings from agent observations. These embeddings proved to be sufficiently informative to support effective training and

achieve strong performance with our meta-agent supervisor. This result highlights the viability of using pretrained visual encoders in environments with image-based observations. Future research could explore the integration of more sophisticated architectures or alternative families of pretrained models. Additionally, fine-tuning or training these models from scratch using data collected directly from the target environments could potentially yield more task-specific and discriminative feature representations, further improving the meta-agent's performance.

CHAPTER 6
Conclusions

To conclude this work, we now present several key takeaways that can be drawn from it. First and foremost, we successfully achieved all the objectives outlined in Chapter **??**. Through this project, we explored the theoretical foundations of Reinforcement Learning and Multi-Agent Reinforcement Learning, which enabled us to characterize cooperative and fully cooperative settings. Building on this theoretical groundwork, we proposed our Generic Multi-Agent Reinforcement Learning (GMARL) framework, which provided the necessary basis to develop our centralized approach featuring the supervisor meta-agent. We also defined two reward aggregation methods—namely, Sum and Softmin + Mean—with the goal of providing the meta-agent with a scalar signal that was sufficiently informative to guide its training and achieve satisfactory results. After establishing the necessary theoretical framework, we proceeded with its implementation in Python, encapsulating the core components of this centralized approach. Subsequently, we conducted a series of experiments which, as discussed in the previous chapter, enabled us to identify the main strengths and limitations of the proposed method. During this experimental phase, we compared the performance of both aggregation strategies, identifying scenarios in which the Softmin + Mean approach yielded more favourable results than simple summation.

These experiments allowed us to identify numerous successful cases of our centralized approach. On the one hand, we concluded that this approach is indeed valid for both cooperative and fully cooperative problems, which is promising for its potential application to other domains and scenarios. Our findings also suggest that, in cooperative settings where agents share a common goal but receive disjoint reward signals, more "intelligent" reward aggregation strategies can be beneficial compared to simple summation. As observed, our Softmin + Mean method achieved superior performance in some cases, as evidenced by the qualitative evaluation of the meta-agent's behaviour through video analysis. We believe this result not only establishes Softmin + Mean as a viable reward aggregation method, but also opens the door to the development of novel aggregation strategies that could potentially outperform our current results. In particular, the exploration of new reward aggregation techniques, possibly in combination with reward shaping methods, may lead to more effective training of the supervisor meta-agent and further improvements in performance.

6.1 Future Work

A potential direction for future work arises from our observation that environments demanding a high level of inter-agent coordination present significant challenges for our model. This highlights the notion that the difficulty of learning in a given environment

may not be determined solely by the structure of the reward functions or the number of agents involved, but also by the inherent dynamics of the environment and the level of coordination required among agents to achieve their goals. It is therefore reasonable to propose future research aimed at determining whether this centralized approach consistently underperforms in environments that require a high degree of coordination and interaction among agents, compared to those where such coordination is less critical. These investigations would help assess whether our model exhibits systematic limitations in highly coupled scenarios, or whether its performance can be improved through specific architectural or methodological adaptations tailored to such settings.

On the other hand, it could be valuable to conduct a series of benchmarks on the PettingZoo environments previously explored, using distributed Multi-Agent Reinforcement Learning algorithms. In this context, a similar approach to the one proposed in [**?**] could be followed, employing distributed algorithms such as *Multi-Agent Proximal Policy Optimization* (MAPPO) [**?**], *Multi-Agent Advantage Actor-Critic* (MAA2C) [**?**], *QMIX* [**?**], or *Counterfactual Multi-Agent Policy Gradients* (COMA) [**?**]. These algorithms typically optimize a separate policy for each agent, in contrast to the centralized approach described in this work, which optimizes a single policy responsible for selecting actions for all agents in the environment. Through such benchmarks, one could assess whether distributed approaches achieve lower, comparable, or superior performance relative to the centralized setup. It would also be worth investigating whether environments in which the centralized method underperforms could benefit from distributed learning. Furthermore, comparing training loop execution times could help quantify the potential computational advantages of centralized methods over distributed ones that optimize as many policies as there are agents.

In conclusion, while our centralized approach based on a supervisor meta-agent presents both strengths and limitations, we regard it as a suitable and promising strategy for addressing Multi-Agent Reinforcement Learning problems through the learning of a single centralized policy. This formulation has the potential to enhance the scalability and efficiency of the learning process by abstracting joint action execution into a sequential decision-making procedure. By compiling the original multi-agent setting into a POMDP, our approach helps mitigate the combinatorial explosion of the joint action space typically encountered in centralized MARL tasks. Although some challenges remain—particularly in tasks requiring high coordination—we have outlined several future directions that may help determine whether the limitations observed are inherent to the model or can be overcome through architectural or methodological variations. Future work could focus on extensive hyperparameter optimization to enhance training stability, exploration of alternative or adaptive reward aggregation functions, and the application of more sophisticated visual representation learning techniques such as fine-tuning pretrained networks on environment-specific data, for image-based environments. Additionally, investigating the behaviour of the approach in highly coordinated or large action-space environments will be crucial to assess and potentially extend its applicability and robustness.

Bibliography

[1] Aso-Mollar, Á. and Onaindia, E. (2024). An Efficient Approach for Cooperative Multi-Agent Learning Problems. 2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI), [online] pp.694–701. doi: https://doi.org/10.1109/ictai62512.2024.00103.

[2] Farama.org. (2023). PettingZoo Documentation. [online] Available at: https://pettingzoo.farama.org/.

[3] Albrecht, S.V., Christianos, F. and Schäfer, L., 2024. Multi-Agent Reinforcement Learning: Foundations and Modern Approaches. Cambridge, MA: MIT Press. Available at: https://www.marl-book.com [Accessed 18 Jun. 2025].

[4] Boutilier, C., 1996. Planning, learning and coordination in multiagent decision processes. University of British Columbia. Available at: http://www.cs.ubc.ca/spider/cebly/craig.html [Accessed 18 Jun. 2025].

[5] Farama.org. (2023). PettingZoo Documentation. [online] Available at: https://pettingzoo.farama.org/environments/butterfly/knights_archers_zombies/.

[6] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V. and Hutter, M. (2019). Learning agile and dynamic motor skills for legged robots. Science Robotics, 4(26), p.eaau5872. doi: https://doi.org/10.1126/scirobotics.aau5872.

[7] OpenAI, Andrychowicz, M., Baker, B., Chociej, M., Jozefowicz, R., McGrew, B., Pachocki, J., Petron, A., Plappert, M., Powell, G., Ray, A., Schneider, J., Sidor, S., Tobin, J., Welinder, P., Weng, L. and Zaremba, W. (2018). Learning Dexterous In-Hand Manipulation. [online] arXiv.org. Available at: https://arxiv.org/abs/1808.00177.

[8] Song, Y., Steinweg, M., Kaufmann, E. and Scaramuzza, D. (n.d.). Autonomous Drone Racing with Deep Reinforcement Learning. [online] Available at: https://rpg.ifi.uzh.ch/docs/IROS21_Yunlong.pdf [Accessed 2 Oct. 2024].

[9] Mao, H., Alizadeh, M., Menache, I. and Kandula, S. (2016). Resource Management with Deep Reinforcement Learning. Proceedings of the 15th ACM Workshop on Hot Topics in Networks - HotNets '16. [online] doi: https://doi.org/10.1145/3005745.3005750.

[10] Xu, Z., Wang, Y., Tang, J., Wang, J. and Gursoy, M. (n.d.). A Deep Reinforcement Learning based Framework for Power-Efficient Resource Allocation in Cloud RANs. [online] Available at: https://raas.syr.edu/wp-content/uploads/2017/12/ICC17.pdf [Accessed 15 Jun. 2025].

[11] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens,

M., Askell, A., Welinder, P., Christiano, P., Leike, J. and Lowe, R. (2022). Training language models to follow instructions with human feedback. [online] Available at: https://arxiv.org/pdf/2203.02155.

[12] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C., Finn, C. and Cz Biohub (n.d.). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. [online] Available at: https://arxiv.org/pdf/2305.18290] [Accessed 27 May 2024].

[13] Samvelyan, M., Rashid, T., de Witt, C.S., Farquhar, G., Nardelli, N., Rudner, T.G.J., Hung, C.-M., Torr, P.H.S., Foerster, J. and Whiteson, S. (2019). The StarCraft Multi-Agent Challenge. arXiv:1902.04043 [cs, stat]. [online] Available at: https://arxiv.org/abs/1902.04043 [Accessed 22 Mar. 2022].

[14] Vinyals, O., Babuschkin, I., Czarnecki, W.M., Mathieu, M., Dudzik, A., Chung, J., Choi, D.H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J.P., Jaderberg, M. and Vezhnevets, A.S. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. Nature, [online] 575(7782), pp.350–354. doi: https://doi.org/10.1038/s41586-019-1724-z.

[15] The AlphaStar team (2019). AlphaStar: Grandmaster level in StarCraft II using multi-agent reinforcement learning. [online] Google DeepMind. Available at: https://deepmind.google/discover/blog/alphastar-grandmaster-level-in-starcraft-ii-using-multi-agent-reinforcement-learning/.

[16] Wang, Z., Zhang, G., Yang, K., Shi, N., Zhou, W., Hao, S., Xiong, G., Li, Y., Sim, M.Y., Chen, X., Zhu, Q., Yang, Z., Nik, A., Liu, Q., Lin, C., Wang, S., Liu, R., Chen, W., Xu, K. and Liu, D. (2023). Interactive Natural Language Processing. [online] arXiv.org. Available at: https://arxiv.org/abs/2305.13246 [Accessed 15 Jun. 2025].

[17] Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X. and Yin, Z. (2023). The Rise and Potential of Large Language Model Based Agents: A Survey. [online] arXiv.org. doi: https://doi.org/10.48550/arXiv.2309.07864.

[18] Park, J., O'brien, J., Cai, C., Morris, M., Liang, P., Bernstein, M., Park, J., O'brien, J., Cai, C., Morris, M., Liang, P. and Bernstein (2023). Generative Agents: Interactive Simulacra of Human Behavior. Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, [online] 23. doi: https://doi.org/10.1145/3586183.3606763.

[19] Chu, T., Wang, J., Codecà, L. and Li, Z. (2019). Multi-Agent Deep Reinforcement Learning for Large-scale Traffic Signal Control. [online] arXiv.org. Available at: https://arxiv.org/abs/1903.04527 [Accessed 12 May 2025].

[20] Zhou, W., Chen, D., Yan, J., Li, Z., Yin, H. and Ge, W. (2022). Multi-agent reinforcement learning for cooperative lane changing of connected and autonomous vehicles in mixed traffic. Autonomous Intelligent Systems, 2(1). doi: https://doi.org/10.1007/s43684-022-00023-5.

[21] Lowe, R., WU, Y., Tamar, A., Harb, J., Pieter Abbeel, OpenAI and Mordatch, I. (2017). Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. Advances in Neural Information Processing Systems, [online] 30. Available at: https://papers.nips.cc/paper_files/paper/2017/hash/68a9750337a418a86fe06c1991a1d64c-Abstract.html [Accessed 16 Jun. 2025].

[22] Zhou, Y., Liu, S., Qing, Y., Chen, K., Zheng, T., Song, J. and Song, M. (2023). Is Centralized Training with Decentralized Execution Framework Centralized Enough for MARL? [online] arXiv.org. Available at: https://arxiv.org/abs/2305.17352 [Accessed 16 Jun. 2025].

[23] Foerster, J., Farquhar, G., Afouras, T., Nardelli, N. and Whiteson, S. (2017). Counterfactual Multi-Agent Policy Gradients. [online] arXiv.org. Available at: https://arxiv.org/abs/1705.08926# [Accessed 16 Jun. 2025].

[24] Barto, A.G., Sutton, R.S. and Anderson, C.W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. IEEE Transactions on Systems, Man, and Cybernetics, SMC-13(5), pp.834–846. doi: https://doi.org/10.1109/tsmc.1983.6313077.

[25] Spaan, M.T.J., 2012. Partially Observable Markov Decision Processes. In: M. Wiering and M. van Otterlo, eds. Reinforcement Learning: State of the Art. Berlin: Springer, pp.387–414.

[26] Oliehoek, F.A. and Amato, C., 2016. A Concise Introduction to Decentralized POMDPs. Springer. Available at: http://www.springer.com/us/book/9783319289274 [Accessed 18 Jun. 2025].

[27] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O., 2017. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347. Available at: https://arxiv.org/abs/1707.06347 [Accessed 18 Jun. 2025]

[28] Boyd, S. and Vandenberghe, L., 2004. Convex Optimization. Cambridge: Cambridge University Press. Available at: https://web.stanford.edu/~boyd/cvxbook/ [Accessed 18 June 2025].

[29] gymnasium.farama.org. (n.d.). Gymnasium Documentation. [online] Available at: https://gymnasium.farama.org/.

[30] Papadopoulos, G., Kontogiannis, A., Papadopoulou, F., Poulianou, C., Koumentis, I. and Vouros, G. (2025). An Extended Benchmarking of Multi-Agent Reinforcement Learning Algorithms in Complex Fully Cooperative Tasks. [online] arXiv.org. Available at: https://arxiv.org/abs/2502.04773 [Accessed 21 Jun. 2025].

[31] He, K., Zhang, X., Ren, S. and Sun, J. (2015). Deep Residual Learning for Image Recognition. [online] arXiv.org. Available at: https://arxiv.org/abs/1512.03385.

[32] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. and Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. 2009 IEEE Conference on Computer Vision and Pattern Recognition, pp.248–255. doi: https://doi.org/10.1109/cvpr.2009.5206848.

[33] Pytorch.org. (2024). torchvision — Torchvision 0.22 documentation. [online] Available at: https://docs.pytorch.org/vision/stable/index.html.

[34] stable-baselines3.readthedocs.io. (n.d.). Stable-Baselines3 Docs - Reliable Reinforcement Learning Implementations — Stable Baselines3 1.2.0a2 documentation. [online] Available at: https://stable-baselines3.readthedocs.io/en/master/.

[35] Pytorch.org. (2024). PyTorch documentation — PyTorch 2.7 documentation. [online] Available at: https://docs.pytorch.org/docs/stable/index.html.

[36] Mordatch, I. and Abbeel, P., 2017. Emergence of grounded compositional language in multi-agent populations. arXiv preprint arXiv:1703.04908.

[37] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P. and Mordatch, I., 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. In: Advances in Neural Information Processing Systems (NeurIPS).

[38] Gupta, J.K., Egorov, M. and Kochenderfer, M., 2017. Cooperative multi-agent control using deep reinforcement learning. In: International Conference on Autonomous Agents and Multiagent Systems. Springer, pp. 66–83.

[39] docs.opencv.org. (n.d.). OpenCV: OpenCV-Python Tutorials. [online] Available at: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html.

[40] kev-usrr (2025). GitHub - kev-usrr/RL_TFM_STUFF. [online] GitHub. Available at: https://github.com/kev-usrr/RL_TFM_STUFF [Accessed 25 Jun. 2025].

[41] Google (2024). colab.google. [online] colab.google. Available at: https://colab.google/.

[42] Arxiv.org. (2025). Self-supervised network distillation: an effective approach to exploration in sparse reward environments. [online] Available at: https://arxiv.org/html/2302.11563v4 [Accessed 8 Jul. 2025].

[43] Burda, Y., Edwards, H., Storkey, A. and Klimov, O. (2018). Exploration by Random Network Distillation. arXiv:1810.12894 [cs, stat]. [online] Available at: https://arxiv.org/abs/1810.12894 [Accessed 10 May 2020].

[44] Ng, A.Y., Harada, D. and Russell, S.J., 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In: Proceedings of the 16th International Conference on Machine Learning (ICML 1999), Bled, Slovenia, 27–30 June 1999. Morgan Kaufmann, pp. 278–287.

[45] Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D. and Meger, D. (2018). Deep Reinforcement Learning That Matters. Proceedings of the AAAI Conference on Artificial Intelligence, 32(1). doi:https://doi.org/10.1609/aaai.v32i1.11694.

[46] Dulac-Arnold, G., Evans, R., van Hasselt, H., Sunehag, P., Lillicrap, T., Hunt, J., Mann, T., Weber, T., Degris, T. and Coppin, B. (2016). Deep Reinforcement Learning in Large Discrete Action Spaces. arXiv:1512.07679 [cs, stat]. [online] Available at: https://arxiv.org/abs/1512.07679 [Accessed 27 Jun. 2025].

[47] Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A. and Wu, Y. (2022). The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games. arXiv:2103.01955 [cs]. [online] Available at: https://arxiv.org/abs/2103.01955.

[48] Rashid, T., Samvelyan, M., Schroeder, C., Farquhar, G., Foerster, J. and Whiteson, S. (2020). Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. [online] arXiv.org. Available at: https://arxiv.org/abs/2003.08839 [Accessed 8 Jul. 2025].